¡University Name¿

¡Department Name¿

PROJECT REPORT

Submitted in partial fulfillment of the requirements
for the award of the degree

PAL: Psychological Anonymity Layers

Submitted by:
¡Your Name¿
Roll No: ¡Roll Number¿

Under the supervision of:
¡Supervisor Name¿

Academic Year: 2025–26

CONTENTS

1 INTRODUCTION **1**

1.1 Motivation 1

1.2 Problem Statement 2

1.3 Objectives 2

1.4 System Summary 2

1.4.1 Extension Subsystem 3

1.4.2 Research Subsystem 3

1.5 Key Features 3

1.6 Scope 4

2 BACKGROUND AND PROBLEM STATEMENT **5**

2.1 Browser Fingerprinting: Technical Background 5

2.2 Why Cookie Blocking is Not Enough 5

2.3 Execution Contexts: Top Frame, Iframes, and Workers 6

2.4 Chrome Extension Constraints: Manifest V3 6

2.5 Problem Statement 7

2.6 Design Goals 7

2.7 Assumptions and Scope Boundaries 7

3 SYSTEM DESIGN AND ARCHITECTURE **9**

3.1 System Overview 9

3.2 High Level Architecture 9

3.3 Extension Subsystem Architecture 10

3.3.1 Service Worker Responsibilities 10

3.3.2 Content Script Split: Main World vs Isolated World 10

3.3.3 Popup UI and Manual Controls 10

3.4 Persona Model and Lifecycle 11

3.4.1 Persona Object Structure 11

3.4.2 Site Scoping 11

3.4.3 Rotation Policy 11

3.5 Operational Modes 12

3.5.1 Vanilla Mode 12

3.5.2 Compatibility Mode 12

3.5.3 Privacy Mode with Epoch Drift . . . 12
3.6 Fingerprint Surface Coverage . . . 12
3.6.1 Navigator and Screen . . . 12
3.6.2 Canvas . . . 13
3.6.3 WebGL . . . 13
3.6.4 Audio . . . 13
3.6.5 WebRTC . . . 13
3.7 Cross Context Propagation Design . . . 14
3.7.1 Iframes . . . 14
3.7.2 Workers . . . 14
3.8 Message Flow and State Management . . . 14
3.9 Research Subsystem Architecture . . . 14
3.9.1 Crawler . . . 14
3.9.2 Probe . . . 15
3.9.3 Offline Analysis . . . 15
3.10 Summary . . . 15

4 IMPLEMENTATION DETAILS 16
4.1 Implementation Goals . . . 16
4.2 Codebase Structure and Module Roles . . . 16
4.3 Mode Routing and Policy Enforcement . . . 17
4.3.1 Surface Enablement . . . 17
4.4 Main World Injection and Hook Installation . . . 17
4.4.1 Why Main World Injection is Necessary . . . 17
4.4.2 Early Hooking Requirement . . . 17
4.5 Context Propagation Implementation . . . 18
4.5.1 Iframe Propagation . . . 18
4.5.2 Worker Propagation . . . 18
4.6 Deterministic Seed Derivation in Implementation . . . 18
4.7 Surface Hooking and Perturbation Logic . . . 19
4.7.1 Canvas Implementation . . . 19
4.7.2 WebGL Implementation . . . 19
4.7.3 Audio Implementation . . . 20
4.7.4 Navigator and Screen Surfaces . . . 20
4.8 Telemetry Collection and Logging . . . 20
4.9 Robustness Handling in the Implementation . . . 21
4.10 Summary . . . 21

5 SYSTEM ARCHITECTURE **22**
5.1 Architecture Overview . . . 22
5.2 Browser Constraint Model . . . 22
5.3 Component Breakdown . . . 23
5.3.1 C1: Configuration and Mode Manager . . . 23
5.3.2 C2: Persona Seed and Epoch Scheduler . . . 23
5.3.3 C3: Main-World Injection Layer . . . 23
5.3.4 C4: Hook Manager and Surface Interceptors . . . 24
5.3.5 C5: Context Propagation Layer . . . 24
5.3.6 C6: Telemetry and Evaluation Pipeline . . . 24
5.4 Control Plane vs Data Plane . . . 24
5.4.1 Control Plane Responsibilities . . . 25
5.4.2 Data Plane Responsibilities . . . 25
5.5 Message Flow and Execution Sequence . . . 25
5.5.1 Top Frame Flow . . . 25
5.5.2 Iframe Flow . . . 26
5.5.3 Worker Flow . . . 26
5.6 Architecture Diagrams . . . 26
5.6.1 System Overview Diagram . . . 27
5.6.2 Injection and Hooking Flow Diagram . . . 28
5.6.3 Epoch and Seed Derivation Diagram . . . 29
5.7 How the Architecture Supports the Invariants . . . 29
5.7.1 Intra-Epoch Stability . . . 30
5.7.2 Inter-Epoch Drift in Privacy Mode . . . 30
5.7.3 Mode Separation . . . 30
5.7.4 Context Robustness . . . 30
5.7.5 Bounded Perturbation . . . 30
5.8 Summary . . . 31

6 EXPERIMENTAL SETUP AND METHODOLOGY **32**
6.1 Purpose of Evaluation . . . 32
6.2 Experimental Design . . . 32
6.2.1 Independent Variables . . . 32
6.2.2 Dependent Variables . . . 32
6.2.3 Experimental Hypotheses . . . 33
6.3 Website Corpus . . . 33
6.4 Automation Framework . . . 33

6.4.1 Crawler Responsibilities . . . 33
6.4.2 Document Start Configuration . . . 34
6.5 Fingerprint Probe Design . . . 34
6.5.1 Probe Goals . . . 34
6.5.2 Contexts Measured . . . 34
6.5.3 Surfaces Extracted . . . 35
6.6 Telemetry Collection . . . 35
6.6.1 Event Format . . . 35
6.6.2 JSONL Storage . . . 35
6.7 Metrics . . . 36
6.7.1 Intra Epoch Stability . . . 36
6.7.2 Inter Epoch Drift . . . 36
6.7.3 Cross Context Consistency . . . 36
6.7.4 Breakage Rate . . . 36
6.8 Controls and Validity Considerations . . . 37
6.8.1 Control Conditions . . . 37
6.8.2 Noise Sources and Mitigation . . . 37
6.8.3 Ethical and Practical Considerations . . . 37
6.9 Summary . . . 38

7 EXPERIMENTAL PROTOCOL AND REPRODUCIBILITY 39
7.1 Why a Formal Protocol is Required . . . 39
7.2 Evaluation Setup Summary . . . 39
7.3 Independent Variables . . . 39
7.3.1 Mode (m) . . . 40
7.3.2 Epoch (e) . . . 40
7.3.3 Context (c) . . . 40
7.3.4 Site (s) . . . 40
7.4 Dependent Variables . . . 40
7.5 Crawl Procedure . . . 40
7.5.1 Step 1: Select Mode and Initialize Policy . . . 40
7.5.2 Step 2: Iterate Epochs . . . 41
7.5.3 Step 3: Site Visit and Context Probing . . . 41
7.5.4 Step 4: Event Capture . . . 41
7.6 Logging and Dataset Construction . . . 41
7.6.1 JSONL Logging Format . . . 41
7.6.2 Required Fields for Reproducibility . . . 42

7.6.3 Dataset Aggregation . . . 42
7.7 Metric Computation Procedure . . . 43
7.7.1 Inter-Epoch Drift . . . 43
7.7.2 Cross-Context Consistency . . . 43
7.7.3 Composite Linkability . . . 43
7.7.4 Error Events . . . 43
7.7.5 Timing Overhead . . . 43
7.8 Reproducibility Checklist . . . 44
7.9 Threats to Validity . . . 44
7.9.1 Limited Site Set . . . 44
7.9.2 Site Variability . . . 44
7.9.3 Browser and Hardware Dependence . . . 44
7.10 Summary . . . 44

8 RESULTS AND ANALYSIS 46
8.1 How to Read This Chapter . . . 46
8.2 Inter-Epoch Drift Rates . . . 46
8.2.1 Canvas Drift . . . 46
8.2.2 WebGL Drift . . . 48
8.2.3 Audio Drift . . . 49
8.2.4 Other Surfaces . . . 51
8.3 Cross-Context Consistency . . . 52
8.3.1 Canvas Consistency . . . 52
8.3.2 WebGL Consistency . . . 52
8.3.3 Navigator Consistency . . . 52
8.3.4 Audio and Screen Consistency . . . 53
8.4 Composite Linkability . . . 53
8.4.1 Top Context . . . 53
8.4.2 Iframe Context . . . 54
8.4.3 Worker Context . . . 54
8.5 Error Signals and Interpretation . . . 55
8.6 Timing Overhead and Performance Cost . . . 57
8.6.1 Top Context . . . 57
8.6.2 Iframe Context . . . 57
8.6.3 Worker Context . . . 57
8.7 Key Findings . . . 58
8.7.1 Finding F1: Privacy Mode Eliminates Strict Linkability in the Top Frame . . . 58

8.7.2 Finding F2: Privacy Mode Strongly Reduces Linkability in Iframes and Workers . . 59
8.7.3 Finding F3: Worker Audio Drift is a Current Coverage Gap 59
8.7.4 Finding F4: Overhead and Errors Identify Engineering Optimization Targets 59
8.8 Summary 59

9 THREAT MODEL 60
9.1 Purpose and Scope 60
9.2 Assets to Protect 60
9.3 Adversary Capabilities 60
9.3.1 C1: Script Execution Capability 60
9.3.2 C2: Access to Fingerprinting APIs 61
9.3.3 C3: Cross-Site Presence 61
9.3.4 C4: Cross-Time Observation 61
9.3.5 C5: Server-Side Storage and Correlation 61
9.3.6 C6: Adaptive Probing 61
9.4 Adversary Goals 62
9.5 Attacker Success Criterion 62
9.5.1 Strict-Equality Composite Key 62
9.5.2 Why Strict Equality is a Reasonable Baseline 62
9.6 Attacker Variants 63
9.6.1 A1: First-Party Fingerprinter 63
9.6.2 A2: Third-Party Embedded Fingerprinter 63
9.6.3 A3: Worker-Focused Fingerprinter 63
9.6.4 A4: Detection-Oriented Fingerprinter 63
9.7 Out of Scope 64
9.7.1 O1: Network-Level Tracking 64
9.7.2 O2: Account-Based Tracking 64
9.7.3 O3: Hardware-Level or OS-Level Side Channels 64
9.7.4 O4: Full Browser Compromise 64
9.8 Mapping the Threat Model to PAL's Mechanisms 64
9.8.1 Against Strict Equality Reuse 64
9.8.2 Against Context Shifting 64
9.8.3 Against Simple Detection by Instability 65
9.9 Summary 65

10 SECURITY ANALYSIS AND ROBUSTNESS 66
10.1 What This Chapter Covers 66
10.2 Detection Analysis 66

10.2.1 D1: Detection via Per-Call Instability . . . 66
10.2.2 D2: Detection via Wrapped Function Signatures . . . 66
10.2.3 D3: Detection via Cross-Context Inconsistency . . . 67
10.2.4 D4: Detection via Timing Artifacts . . . 67
10.3 Bypass Analysis . . . 68
10.3.1 B1: Context Shifting . . . 68
10.3.2 B2: Surface Substitution . . . 68
10.3.3 B3: Fuzzy Matching Rather Than Strict Equality . . . 68
10.4 Residual Linkability: What Still Links the User . . . 69
10.5 Robustness Properties and Engineering Hardening . . . 69
10.5.1 R-API: Preserve API Contracts . . . 69
10.5.2 R-Bounds: Bound Perturbations . . . 69
10.5.3 R-Determinism: Enforce Intra-Epoch Stability . . . 70
10.5.4 R-Fail-Safe: Safe Fallback on Exceptions . . . 70
10.5.5 R-Observability: Logging Integrity . . . 70
10.6 Hardening Roadmap . . . 70
10.7 Summary . . . 70

11 PROBLEM DEFINITION AND FORMAL METRICS 71
11.1 Why Fingerprinting Remains a Tracking Primitive . . . 71
11.2 Core Entities and Notation . . . 71
11.2.1 Modes . . . 71
11.2.2 Sites . . . 71
11.2.3 Epochs . . . 71
11.2.4 Execution Contexts . . . 71
11.2.5 Fingerprint Surfaces . . . 72
11.2.6 Fingerprint Output . . . 72
11.3 Composite Fingerprint Key . . . 72
11.4 Problem Statement . . . 72
11.4.1 Tracking via Stable Fingerprints . . . 72
11.4.2 PAL Objective . . . 73
11.5 Formal Metrics . . . 73
11.5.1 Inter-Epoch Drift Rate . . . 73
11.5.2 Cross-Context Consistency Rate . . . 73
11.5.3 Composite Linkability . . . 74
11.5.4 Error Signal Metrics . . . 74
11.5.5 Timing Overhead Metrics . . . 75

11.6 Success Criteria by Mode . . . 75
11.6.1 Vanilla . . . 75
11.6.2 Compatibility Mode . . . 75
11.6.3 Privacy Mode . . . 75
11.7 Summary . . . 75

12 DESIGN PRINCIPLES AND SYSTEM INVARIANTS 77
12.1 Why Design Principles Matter for a Fingerprinting Defense . . . 77
12.2 High-Level Design Principles . . . 77
12.2.1 DP1: Composite Tracking Requires Composite Defenses . . . 77
12.2.2 DP2: Drift Must Be Controlled, Not Random . . . 77
12.2.3 DP3: Persona Coherence is a First-Class Constraint . . . 78
12.2.4 DP4: Context Coverage is Required . . . 78
12.2.5 DP5: Deployability Under Real Browser Constraints . . . 78
12.3 System Invariants . . . 78
12.3.1 Invariant I1: Intra-Epoch Stability . . . 78
12.3.2 Invariant I2: Inter-Epoch Drift for Selected Surfaces in Privacy Mode . . . 79
12.3.3 Invariant I3: Mode Separation . . . 79
12.3.4 Invariant I4: Persona Coherence Across Surfaces . . . 79
12.3.5 Invariant I5: Context Robustness . . . 79
12.3.6 Invariant I6: Bounded Perturbations . . . 80
12.4 Non-Goals as Design Constraints . . . 80
12.4.1 NG1: Perfect Anonymity . . . 80
12.4.2 NG2: Defeating Dedicated Anti-Fraud Fingerprinting at All Costs . . . 80
12.4.3 NG3: Breaking Web Compatibility for Maximum Drift . . . 80
12.5 Design Tradeoffs and Rationale . . . 80
12.5.1 Tradeoff T1: Drift Strength Versus Plausibility . . . 80
12.5.2 Tradeoff T2: Context Uniformity Versus Deployment Complexity . . . 80
12.5.3 Tradeoff T3: Performance Versus Coverage . . . 81
12.6 How These Invariants Will Be Validated . . . 81
12.7 Summary . . . 81

13 DETERMINISTIC DRIFT MECHANISM 82
13.1 Why Deterministic Drift is the Core Contribution . . . 82
13.2 Design Requirements . . . 82
13.2.1 R1: Intra-Epoch Determinism . . . 82
13.2.2 R2: Inter-Epoch Separation . . . 82
13.2.3 R3: Surface Separation . . . 83

13.2.4 R4: Context Awareness . . . 83
13.2.5 R5: Boundedness and Plausibility . . . 83
13.3 Seed Hierarchy . . . 83
13.3.1 Base Persona Seed . . . 83
13.3.2 Epoch Identifier . . . 83
13.3.3 Surface Label and Context Key . . . 83
13.3.4 Derived Surface Seed . . . 84
13.4 Optional Stabilizers and Subkeys . . . 84
13.5 Deterministic Pseudorandom Number Generation . . . 84
13.6 Perturbation Construction Strategy . . . 85
13.6.1 Step 1: Derive Seed . . . 85
13.6.2 Step 2: Initialize PRNG . . . 85
13.6.3 Step 3: Generate Perturbation Parameters . . . 85
13.6.4 Step 4: Apply Perturbation at Hook Sites . . . 85
13.7 Surface-Specific Strategies . . . 86
13.7.1 Canvas Drift Strategy . . . 86
13.7.2 WebGL Drift Strategy . . . 86
13.7.3 Audio Drift Strategy . . . 87
13.8 Preventing Common Failure Modes . . . 87
13.8.1 FM1: Per-Call Randomness Signatures . . . 87
13.8.2 FM2: Unbounded or Invalid Outputs . . . 87
13.8.3 FM3: Cross-Surface Correlation Artifacts . . . 87
13.8.4 FM4: Context Collapse . . . 87
13.9 Summary . . . 88

14 DISCUSSION, LIMITATIONS, AND FUTURE WORK 89
14.1 Discussion Framed by the Threat Model . . . 89
14.2 Interpreting the Core Results . . . 89
14.2.1 Why Top-Frame Linkability Reaches Zero . . . 89
14.2.2 Why Iframe Linkability Remains Non-Zero . . . 90
14.2.3 Why Worker Linkability is Very Low Despite Worker Audio Gap . . . 90
14.3 Limitations . . . 90
14.3.1 L1: Worker Audio Drift is Not Yet Implemented End-to-End . . . 90
14.3.2 L2: Cross-Context Coherence is Not Enforced for Some Drifted Surfaces . . . 91
14.3.3 L3: Performance Overhead is Non-Trivial and Highly Variable . . . 91
14.3.4 L4: Error Events Increase Under Hooking . . . 91
14.3.5 L5: Dataset Size and Coverage . . . 91

14.4 Future Work Roadmap . . . 92
14.4.1 FW1: Worker Audio Instrumentation . . . 92
14.4.2 FW2: Improve Iframe Drift Uniformity . . . 92
14.4.3 FW3: Optional Cross-Context Coherent Privacy Profile . . . 92
14.4.4 FW4: Performance Optimization . . . 93
14.4.5 FW5: Expand Evaluation Scale and Add Confidence Reporting . . . 93
14.5 Broader Implications . . . 93
14.6 Summary . . . 94

15 CONCLUSION **95**
15.1 Summary of the Problem . . . 95
15.2 What PAL Set Out To Do . . . 95
15.3 Core Contributions . . . 95
15.3.1 Contribution 1: Deterministic Epoch-Based Drift as a Deployable Defense Primitive 95
15.3.2 Contribution 2: Multi-Context Evaluation as a Required Standard . . . 95
15.3.3 Contribution 3: A Metric Framework Centered on Composite Linkability . . . 96
15.4 Empirical Results and Supported Claims . . . 96
15.5 What This Means Practically . . . 96
15.6 Next Engineering Milestones . . . 96
15.7 Closing Statement . . . 97

16 RELATED WORK **98**
16.1 Why Related Work Matters Here . . . 98
16.2 Fingerprinting Surfaces and Measurement Practices . . . 98
16.3 Randomization and Noise Injection Defenses . . . 98
16.4 Standardization and Entropy Reduction . . . 99
16.5 Partitioning and Isolation Strategies . . . 99
16.6 Anti-Fingerprinting in Privacy-Focused Browsers . . . 100
16.7 How PAL Fits . . . 100
16.8 Summary . . . 100

17 LIMITATIONS AND ETHICAL CONSIDERATIONS **102**
17.1 Scope of Evaluation . . . 102
17.2 Surface Coverage and Design Tradeoffs . . . 102
17.3 Interpretation of Error and Performance Signals . . . 102
17.4 Ethical Considerations . . . 102
17.5 Responsible Disclosure and Deployment . . . 103

18 APPENDICES **104**

A METRIC DEFINITIONS AND COMPUTATION DETAILS **105**

A.1 Notation . . . 105
A.2 Inter-Epoch Drift Rate . . . 105
A.2.1 Definition . . . 105
A.2.2 Interpretation . . . 106
A.3 Cross-Context Consistency Rate . . . 106
A.3.1 Definition . . . 106
A.3.2 Interpretation . . . 106
A.4 Composite Linkability Rate . . . 106
A.4.1 Composite Key Construction . . . 106
A.4.2 Definition . . . 106
A.4.3 Interpretation . . . 107
A.5 Error Metrics . . . 107
A.5.1 Error Event Count . . . 107
A.5.2 Sites with Any Error . . . 107
A.6 Timing Metrics . . . 107

B EXPERIMENTAL CONFIGURATION AND REPRODUCIBILITY CHECKLIST **109**

B.1 Environment . . . 109
B.2 PAL Build and Versioning . . . 109
B.3 Policy Configuration . . . 109
B.4 Crawl Settings . . . 109
B.5 Output Artifacts . . . 110
B.6 Reproduction Steps . . . 110

C SITE LIST **111**

C.1 Sites Used in the Crawl . . . 111

D ERROR TAXONOMY AND INTERPRETATION **112**

D.1 Why a Taxonomy Helps . . . 112
D.2 Proposed Error Categories . . . 112
D.2.1 E1: Baseline Third-Party Script Errors . . . 112
D.2.2 E2: API Contract Violations . . . 112
D.2.3 E3: Security Exceptions . . . 112
D.2.4 E4: Feature-Detection Mismatches . . . 112
D.2.5 E5: CSP and Policy Violations . . . 112

D.3 How to Use the Taxonomy 113
D.4 What This Report Currently Shows 113

E REFERENCES **115**

LIST OF FIGURES

3.1 High level architecture of PAL showing extension components and the evaluation pipeline. . 9

3.2 Persona delivery and enforcement message flow under MV3. 14

5.1 High-level PAL system overview showing control plane, data plane, and evaluation pipeline. 27

5.2 Injection strategy and hook installation flow from extension world to main world across contexts. 28

5.3 Conceptual seed derivation from persona seed, epoch, surface label, and context. 29

8.1 Canvas inter-epoch drift rates by context for vanilla, compatibility, and privacy modes. . . . 48

8.2 WebGL inter-epoch drift rates by context for vanilla, compatibility, and privacy modes. . . . 49

8.3 Audio inter-epoch drift rates by context for vanilla, compatibility, and privacy modes. Worker-audio drift is currently absent, motivating future work on worker audio instrumentation. . . . 51

8.4 Composite fingerprint linkability across epochs by context and mode. Privacy mode eliminates strict-equality linkability in the top frame and sharply reduces it in iframe and worker contexts. 55

8.5 Total captured error events by mode. Vanilla also exhibits baseline errors, indicating that error counts should be interpreted as a robustness signal rather than a definitive breakage measure. 56

8.6 Probe timing (mean and std) by context and mode. Compatibility and privacy modes introduce overhead relative to vanilla; variability is highest in iframe and worker contexts. 58

LIST OF TABLES

CHAPTER 1

INTRODUCTION

Modern computer systems increasingly assist users while they create content. IDEs provide live linting, browsers enforce security policies, and modern platforms add real time checks for correctness and consistency. In the privacy domain, websites and third party scripts can also measure device and browser characteristics through browser fingerprinting. This creates a tracking channel that can persist even when cookies are blocked or frequently cleared, because the identifier is computed from observable execution and rendering behavior rather than stored state.

This project builds **PAL (Psychological Anonymity Layers)**, a Chrome Manifest V3 extension that reduces fingerprint based linkability by introducing **site scoped synthetic personas** and enforcing those personas across multiple fingerprint surfaces. The system is not a browser fork. It is a practical, deployable extension that runs at `document_start` and modifies fingerprint exposed APIs in the page execution environment. Alongside the extension, PAL includes a research pipeline that crawls websites, extracts comparable fingerprint vectors, and measures stability, drift, and functional breakage signals under controlled experimental conditions.

The result is a complete system that combines persona based identity abstraction, multi surface API enforcement, controlled drift for unlinkability, and an empirical evaluation workflow into a single end to end project.

1.1 Motivation

Browser fingerprinting is attractive to trackers because it requires no persistent storage and can be computed during a single page load. A fingerprint is typically a high dimensional vector formed from a combination of:

- **Device and browser attributes:** user agent strings, platform, languages, hardware concurrency, device memory, and screen characteristics.
- **Rendering outputs:** Canvas and WebGL values that depend on GPU, drivers, fonts, and rendering pipeline behavior.
- **Signal processing behavior:** Audio fingerprinting that relies on numerical differences in audio pipelines.
- **Network exposed information:** local and public addressing patterns, often revealed through WebRTC candidates.

Privacy defenses face a practical engineering tension:

- Strong changes improve unlinkability but can break sites or trigger detection.
- Conservative changes preserve compatibility but keep identifiers stable across time.

- A defense must cover multiple execution contexts because fingerprinting can run in the top page, iframes, and workers.

PAL is motivated by the need for a system that can explicitly expose and measure this tradeoff, instead of relying on unverified claims.

1.2 Problem Statement

The problem addressed in this project is:

> How can a browser side defense reduce fingerprint linkability by enforcing coherent identity signals, while enabling controlled drift over time and quantifying the resulting compatibility cost across real websites?

This breaks into four technical requirements:

- **Coherent personas:** construct synthetic identities whose attributes do not contradict each other.
- **Multi surface enforcement:** ensure key fingerprint surfaces reflect the persona consistently.
- **Cross context behavior:** reduce gaps between top frame, iframe, and worker contexts.
- **Measurable outcomes:** produce repeatable empirical evidence for stability, drift, and breakage.

1.3 Objectives

PAL is designed with the following objectives:

- Assign each website a persona that acts as the single source of truth for exposed identity signals.
- Intercept and modify fingerprint relevant APIs in the main world at `document_start`.
- Provide two operational modes to study the stability vs unlinkability tradeoff.
- Implement deterministic drift in privacy mode so behavior is stable within an epoch and changes across epochs.
- Support reproducible experiments through an automated crawler, structured telemetry, and offline analysis.

1.4 System Summary

PAL is composed of an extension subsystem and a research subsystem.

1.4.1 Extension Subsystem

The extension includes:

- A background service worker that maintains persona state and applies a rotation policy.
- A main world hook script that intercepts fingerprint surfaces and enforces persona aligned outputs.
- A loader script in an isolated world that securely fetches the persona from the extension and passes it to the main world.
- A popup interface for inspecting status and manually rotating identities.

1.4.2 Research Subsystem

The research pipeline includes:

- An automated crawler that runs websites under controlled mode and epoch settings.
- A fingerprint probe that generates comparable vectors for top frame, iframe, and worker contexts.
- A structured logging format that records telemetry, vectors, and breakage signals.
- Analysis scripts that verify invariants and quantify drift and consistency properties.

1.5 Key Features

The main features implemented in PAL are:

- **Persona based identity abstraction** enforced across Navigator and Screen surfaces, with configuration and lifecycle managed by the background service worker.
- **Canvas protections** including hooks for `getImageData`, `toDataURL`, and selected text measurement paths, with optional deterministic perturbation.
- **WebGL protections** including readback interception and parameter spoofing for key renderer identifiers.
- **Audio protections** that introduce controlled perturbations in sample extraction paths under privacy mode.
- **WebRTC candidate filtering** to reduce exposure of local and reflexive addressing patterns.
- **Worker handling** through interception of worker construction and injection of a bootstrap layer so protections extend beyond the main window context.
- **Controlled drift across epochs** to reduce long term linkability while maintaining intra epoch consistency.

1.6 Scope

This report focuses on fingerprint mitigation within the constraints of a Chrome Manifest V3 extension and its evaluation pipeline. The scope includes persona construction, API hooking across major fingerprint surfaces, cross context handling, and empirical measurement of stability, drift, and breakage signals.

The scope does not include modifying the browser engine, guaranteeing protection against server side correlation or account based tracking, or proving complete undetectability against adaptive adversaries.

CHAPTER 2

BACKGROUND AND PROBLEM STATEMENT

2.1 Browser Fingerprinting: Technical Background

Browser fingerprinting is a tracking method in which a website computes an identifier from observable properties of the browser, device, and execution environment. Unlike cookies, the identifier is not stored on the client. It is reconstructed on demand using signals that are available to JavaScript and browser APIs. A fingerprint is typically formed by concatenating or hashing a set of features and then using the resulting value for recognition across page loads, sessions, and sometimes across different sites.

In practice, fingerprinting works because many signals are either stable by design or only vary within a small range. Even when a single attribute is not unique, the combined vector becomes distinguishing. Common classes of signals include:

- **Navigator and platform properties:** `navigator.userAgent`, `navigator.platform`, `navigator.languages`, CPU core count, memory hints, and vendor fields.
- **Screen and display properties:** `screen.width`, `screen.height`, color depth, device pixel ratio, and viewport related values.
- **Graphics pipeline properties:** WebGL parameters (vendor, renderer, supported extensions) and pixel outputs from rendering pipelines.
- **Canvas rendering outputs:** image extraction through `getImageData` and serialization via `toDataURL`, which often reflect subtle differences in font rasterization and GPU acceleration.
- **Audio pipeline behavior:** sample arrays extracted from offline rendering or buffer reads, which can differ due to floating point behavior and implementation details.
- **Network exposures:** IP and local network hints often obtainable through WebRTC candidate gathering.

A tracker does not need to rely on one surface. Modern scripts collect from many surfaces and use redundancy. If one surface is blocked or spoofed, the tracker can still use the remaining ones. This is why defenses that only modify a single API often fail in real settings.

2.2 Why Cookie Blocking is Not Enough

Cookie policy changes and third party cookie restrictions reduce traditional tracking, but fingerprinting persists for three reasons:

- The fingerprint can be computed without writing anything to storage.
- It can be computed early, even before user interaction, during initial page load.

- It can be computed repeatedly and refreshed, making it robust to periodic resets.

Even if users clear cookies frequently, the fingerprint based identifier can re link sessions, restore server side profiles, and reconnect identity across different browsing contexts. This makes fingerprinting a direct privacy risk even when standard cookie defenses are applied.

2.3 Execution Contexts: Top Frame, Iframes, and Workers

A key engineering issue in fingerprint mitigation is that fingerprinting does not only run in the main page context. Scripts can execute in:

- **Top frame (window):** the primary document context where most scripts execute.
- **Iframes:** embedded contexts that can be same origin or cross origin depending on the embedding.
- **Workers:** background contexts that execute JavaScript off the main thread, including Dedicated Workers and sometimes Shared Workers.

Defenses that only patch the top frame often leave gaps. A tracker can load a probe script in an iframe or a worker and compute a fingerprint from that isolated environment. In addition, some surfaces are accessed more commonly inside workers for performance reasons, especially in WebGL and Canvas workflows when sites use offscreen rendering.

Therefore, a practical defense must consider not only which surfaces to patch, but also **where** to patch them. Cross context consistency also matters. If the top frame shows one identity and an iframe shows a different identity, the mismatch itself can become a detection signal.

2.4 Chrome Extension Constraints: Manifest V3

PAL is implemented as a Chrome Manifest V3 extension. MV3 introduces constraints that affect fingerprint defenses:

- The persistent background page is replaced with a **service worker**, which can be suspended and restarted.
- Many actions must be designed around message passing and event based activation.
- Content scripts run in an **isolated world** by default, which does not directly modify the page JavaScript environment.

Because fingerprinting scripts run in the page's JavaScript environment, a defense that wants to intercept prototypes and core functions must operate in the **main world**. Under MV3, this typically requires a design where:

- A main world script installs hooks early.
- An isolated world script communicates with the extension service worker to obtain configuration and persona state.
- A secure bridge passes the persona data into the main world so the hook logic can enforce it.

PAL is designed around this MV3 reality rather than relying on privileged browser modifications.

2.5 Problem Statement

The project addresses the following problem:

> Websites can compute stable, high dimensional fingerprints from multiple browser surfaces and multiple execution contexts. A practical defense must reduce fingerprint linkability while preserving website compatibility, and it must be measurable under repeatable experiments.

From an engineering perspective, this means the defense should satisfy four properties:

1. **Coherence:** outputs across different APIs should be mutually consistent and aligned with one plausible identity.
2. **Coverage:** the defense should apply across multiple high value fingerprint surfaces.
3. **Cross context operation:** the defense should apply to top frame, iframes, and workers without producing contradictions.
4. **Quantifiable tradeoff:** the defense should provide evidence about drift, stability, and breakage rather than relying on qualitative claims.

2.6 Design Goals

PAL is built with the following design goals:

1. **Persona based abstraction:** represent the identity as a structured persona object and use it as the single source of truth.
2. **Two explicit modes:** provide a compatibility oriented mode and a privacy oriented mode so the tradeoff can be studied directly.
3. **Deterministic drift:** in privacy mode, implement controlled drift across epochs while maintaining stability within the same epoch.
4. **Main world enforcement:** intercept and modify fingerprint APIs in the page execution environment at `document_start`.
5. **Research driven validation:** integrate a crawling and probing pipeline so stability, drift, and breakage can be measured across real websites.

2.7 Assumptions and Scope Boundaries

PAL is a browser side defense focused on fingerprint mitigation. The system does not attempt to solve every form of tracking. The following scope boundaries are assumed:

- The system does not prevent tracking based on user logins, account identifiers, or server side correlation of activity.

- The system does not claim perfect undetectability against a powerful adversary that actively searches for extension artifacts.
- The system focuses on reducing linkability through known fingerprint surfaces and does not attempt to eliminate all possible side channels.

These boundaries are important for a correct evaluation. The project goal is not to claim complete invisibility, but to build a realistic defense and to measure how it performs under controlled modes and across many websites.

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

3.1 System Overview

PAL (Psychological Anonymity Layers) is designed as a two part system:

- **Extension subsystem:** a Chrome Manifest V3 extension that enforces a synthetic persona across fingerprint relevant browser surfaces by injecting hooks at `document_start`.
- **Research subsystem:** an automated measurement pipeline that runs controlled experiments over websites, extracts fingerprint vectors from multiple contexts, and records structured telemetry for analysis.

The system is not built as a browser fork. This design decision keeps deployment practical and makes it possible to evaluate PAL in environments similar to real users. The core engineering challenge is to reliably modify fingerprint exposed behavior seen by page scripts, while still operating inside the constraints of MV3.

3.2 High Level Architecture

PAL uses four primary extension components and a separate research stack:

- **Service worker (background):** stores persona state, applies policy, and serves persona data to tabs.
- **Main world hook layer:** injected at `document_start` to intercept target APIs before website scripts execute.
- **Isolated world loader:** runs as a normal content script to communicate with the extension and deliver persona data to the main world.
- **Popup UI:** provides visibility into the active mode and persona and allows manual persona rotation.
- **Research pipeline:** crawler + probe + analysis scripts that automate experiments and validate outcomes.

Figure 3.1 shows the relationships between these components.

Figure Placeholder: PAL Architecture Overview
Insert a block diagram with: Website Page (top/iframe/worker) on the left, Main World Hook Layer and Loader in the middle, Service Worker and Storage on the right, Popup as control, and Research Pipeline below.

Figure 3.1: High level architecture of PAL showing extension components and the evaluation pipeline.

3.3 Extension Subsystem Architecture

3.3.1 Service Worker Responsibilities

Under MV3, the background environment is a service worker. PAL assigns the service worker the following responsibilities:

1. **Persona store:** maintain a mapping between a site key (for example, eTLD+1 or origin) and a persona object.
2. **Policy enforcement:** decide whether a persona should be reused, refreshed, or rotated based on configuration, time, or explicit user action.
3. **Messaging hub:** respond to requests from content scripts for persona delivery and status, and from the popup UI for manual rotation.
4. **State persistence:** store personas and metadata (timestamps, counters, last rotation) using extension storage so state survives service worker suspension.

A crucial design constraint is that the service worker can be stopped when idle. Therefore, the persona lifecycle design must tolerate re initialization. PAL solves this by treating storage as the source of truth and by making all persona retrieval a message driven action initiated by content scripts.

3.3.2 Content Script Split: Main World vs Isolated World

Chrome MV3 content scripts execute in an isolated world by default. However, fingerprinting scripts run in the page's JavaScript environment. If PAL only executed in the isolated world, the hooks would not be visible to page scripts, and the defense would fail.

Therefore PAL uses a split design:

- **Main world hook layer:** injected with `world=MAIN` at `document_start`. This script installs hooks on prototypes and functions that websites call to compute fingerprint signals.
- **Isolated world loader:** injected at `document_start` in the isolated world. This script can safely call `chrome.runtime.sendMessage` to obtain persona data from the service worker. It then bridges the persona to the main world.

The bridge is implemented using a minimal message protocol from isolated to main world, for example using `window.postMessage`. This avoids exposing extension APIs directly to the page.

3.3.3 Popup UI and Manual Controls

PAL includes a popup UI that supports two operational needs:

- **Observability:** show active mode, persona identifier, and key persona fields (platform, user agent family, screen).
- **Control:** allow the user to rotate the persona for the current site or apply a shift operation immediately.

This component interacts only with the service worker and does not directly modify the page. That separation keeps enforcement centralized and reduces UI coupling with hook logic.

3.4 Persona Model and Lifecycle

3.4.1 Persona Object Structure

PAL represents identity using a structured persona object. The persona includes fields that cover multiple fingerprint surfaces. Typical fields include:

- **Identity metadata:** persona ID, friendly name, creation timestamp.
- **Navigator attributes:** user agent, platform, hardware concurrency, device memory, language and languages, optional user agent data behavior.
- **Screen attributes:** width, height, color depth, pixel depth, and other display properties.
- **Graphics attributes:** WebGL vendor and renderer identifiers where applicable.
- **Noise control:** a seed used as the deterministic basis for drift and perturbation in privacy mode.

The persona acts as the single source of truth. Hooked surfaces either return persona values directly (for scalar properties) or use the seed to generate deterministic perturbations (for array or pixel outputs).

3.4.2 Site Scoping

PAL must choose how to scope personas, because scoping affects both privacy and usability.

- **Per origin scoping** isolates identities per origin, improving privacy but potentially breaking cross origin flows used by legitimate websites.
- **Per top level site scoping** (eTLD+1) reduces breakage when subdomains are used for the same service, but it may allow some correlation across subdomains.

PAL is designed to allow site scoping to be configured by policy. In evaluation mode, scoping can be fixed for controlled experiments.

3.4.3 Rotation Policy

PAL supports persona rotation in two forms:

- **Policy driven rotation:** rotate after a time threshold, after a certain number of visits, or at epoch boundaries in privacy experiments.
- **Manual rotation:** a user triggered identity shift for the current site via the popup UI.

Rotation changes the persona seed and therefore changes derived signals. However, PAL also supports a compatibility mode where the persona is stable and does not drift, enabling stable browsing where unlinkability is not prioritized.

3.5 Operational Modes

PAL defines two core operational modes, plus a baseline mode used in evaluation.

3.5.1 Vanilla Mode

Vanilla mode is a baseline where no modifications are applied. It is used in experiments to capture the natural fingerprint behavior of the browser.

3.5.2 Compatibility Mode

Compatibility mode enforces a persona but avoids drift across time. The goals are:

- Create coherent outputs across surfaces.
- Reduce contradictions and anomalies caused by partial spoofing.
- Maintain stable values so site functionality is preserved.

In this mode, the persona seed is constant for a site until rotation policy explicitly changes it.

3.5.3 Privacy Mode with Epoch Drift

Privacy mode introduces controlled drift across epochs. An epoch can be defined by a time window or by an experiment counter. The goals are:

- Maintain stability within a single epoch so a page session behaves consistently.
- Ensure drift across epochs so long term linkability decreases.
- Avoid per call randomness that can create inconsistent outputs and increase detectability.

In this mode, PAL derives perturbations deterministically from the persona seed and epoch. This yields repeatable behavior for testing and ensures that drift is not arbitrary noise.

3.6 Fingerprint Surface Coverage

PAL targets a set of surfaces commonly used in fingerprint scripts.

3.6.1 Navigator and Screen

Navigator and screen properties provide coarse but widely collected identity signals. PAL enforces persona aligned values for:

- `navigator.userAgent`, `navigator.platform`
- `navigator.languages`, `navigator.language`

- hardware hints such as `navigator.hardwareConcurrency` and `navigator.deviceMemory` when available
- screen fields such as width, height, and depth

These are enforced through property descriptor overrides on prototypes where possible.

3.6.2 Canvas

Canvas fingerprinting typically extracts pixel buffers or serializes images. PAL hooks common extraction functions such as:

- `CanvasRenderingContext2D.getImageData`
- `HTMLCanvasElement.toDataURL`
- Offscreen canvas equivalents where present

In privacy mode, PAL applies deterministic perturbations to extracted pixel buffers based on the persona seed and epoch. In compatibility mode, it can return consistent outputs aligned with the persona and avoid drift.

3.6.3 WebGL

WebGL exposes high entropy signals through GPU behavior and pixel readback. PAL targets:

- Parameter queries that reveal vendor and renderer.
- Pixel extraction such as `readPixels`.

As with canvas, privacy mode introduces controlled drift, while compatibility mode aims to keep consistent persona aligned outputs.

3.6.4 Audio

Audio fingerprints commonly rely on floating point differences in offline rendering. PAL targets audio sample extraction methods such as:

- `AudioBuffer.getChannelData`
- Related extraction paths such as copy based reads

In privacy mode, PAL injects deterministic perturbation into the sample arrays so linkability decreases across epochs.

3.6.5 WebRTC

WebRTC ICE candidate gathering can expose local IP addresses and network structure. PAL reduces this exposure by filtering candidates in the connection process based on policy. In privacy mode the filtering can be more aggressive.

3.7 Cross Context Propagation Design

3.7.1 Iframes

PAL ensures the hook layer executes in all frames by using content script injection configuration that includes subframes. This ensures that if a fingerprint script runs in an iframe, it will encounter the same enforcement logic and persona.

3.7.2 Workers

Workers introduce a different constraint. Worker code runs in a separate global scope and does not automatically receive hooks installed in the window environment. PAL addresses this by intercepting worker construction and injecting a bootstrap layer that loads hook logic and persona state before the worker executes its target script.

This design is essential for defending against trackers that fingerprint from background contexts and also for ensuring that the evaluation probe can measure consistent behavior across contexts.

3.8 Message Flow and State Management

Figure 3.2 summarizes the message sequence used to deliver persona state and enforce it in the page.

Figure Placeholder: Message Flow
(1) Loader content script sends persona request to service worker.
(2) Service worker returns persona + mode + epoch.
(3) Loader posts message to main world hook layer.
(4) Hook layer caches persona and installs/updates hooks.
(5) Popup UI can request status or trigger rotation via service worker.

Figure 3.2: Persona delivery and enforcement message flow under MV3.

State is managed with three goals:

- Ensure the hook layer has persona data before fingerprint scripts run.
- Persist persona mappings across service worker suspension.
- Keep persona consistent across frames and contexts under the chosen scoping policy.

3.9 Research Subsystem Architecture

PAL includes a research pipeline designed to evaluate the defense empirically.

3.9.1 Crawler

The crawler automates visits to a list of websites and runs them under controlled configurations. It sets:

- mode (vanilla, compatibility, privacy)
- epoch identifier
- run identifier and site key

The crawler also captures structured console telemetry and stores it in a machine readable format for offline processing.

3.9.2 Probe

The probe script computes fingerprint vectors in:

- top level context
- iframe context
- worker context

The probe extracts comparable values from canvas, WebGL, audio, navigator, and screen surfaces, hashes them into compact identifiers, and emits them as structured telemetry.

3.9.3 Offline Analysis

The analysis stage validates invariants and computes metrics such as:

- intra epoch stability
- inter epoch drift
- cross context consistency
- error and breakage rates

This pipeline ensures that PAL can be evaluated with repeatable experiments and supports reporting results based on collected evidence.

3.10 Summary

This chapter presented the full system architecture of PAL, covering the MV3 extension design, persona lifecycle, operational modes, and the research pipeline used for empirical evaluation. The next chapter describes implementation details, including how hooks are installed, how deterministic drift is generated, and how worker and iframe propagation are implemented in practice.

CHAPTER 4

IMPLEMENTATION DETAILS

4.1 Implementation Goals

The implementation of PAL is guided by the design principles and invariants established earlier. Concretely, the implementation must achieve the following:

- **Implementable deployment:** PAL must work as a real extension-based tool under browser constraints.
- **Mode separation:** vanilla, compatibility, and privacy must be meaningfully distinct and configurable.
- **Hook correctness:** interception must preserve API contracts and expected types.
- **Deterministic drift:** privacy mode must drift across epochs while remaining stable within an epoch.
- **Context coverage:** behavior must apply across top, iframe, and worker contexts to the extent possible.
- **Traceable evaluation:** all measurements must be loggable in a structured format so metrics can be computed reliably.

This chapter explains how PAL is implemented to satisfy these goals.

4.2 Codebase Structure and Module Roles

PAL is organized into functional layers that align with the architecture:

- **Policy layer:** defines mode, drift settings, enabled surfaces, persona configuration.
- **Seed and epoch layer:** generates the persona seed and provides the epoch identifier and derived surface seeds.
- **Injection layer:** ensures that hook logic runs in the correct execution world and loads early enough.
- **Hook layer:** implements surface interceptors and routes each hook through mode-specific behavior.
- **Perturbation layer:** contains surface-specific deterministic transformations.
- **Telemetry layer:** emits events, measurement vectors, timing samples, and error signals for evaluation.

Even if the exact folder names differ across versions, this layered mental model is important because it clarifies why the system is testable and extensible.

4.3 Mode Routing and Policy Enforcement

All hooks route through a single decision pattern:

1. Identify the active mode $m \in \{\text{vanilla}, \text{compat}, \text{privacy}\}$.
2. If vanilla: bypass interception and return original outputs.
3. If compat: apply persona coherence logic but disable drift.
4. If privacy: apply persona coherence constraints plus deterministic drift for selected surfaces.

This routing logic is implemented as a centralized policy decision function rather than being duplicated per surface. Centralizing mode decisions reduces bugs and ensures consistent semantics.

4.3.1 Surface Enablement

PAL allows surfaces to be enabled or disabled independently. This provides two benefits:

- It allows experimental evaluation by turning surfaces on/off and observing linkability changes.
- It supports practical deployment tradeoffs where a user may prefer fewer hooks for performance reasons.

In the evaluation setup, the primary high-entropy surfaces (Canvas, WebGL, Audio) are the key levers for reducing linkability.

4.4 Main World Injection and Hook Installation

4.4.1 Why Main World Injection is Necessary

Many browser APIs used for fingerprinting exist in the page's main JavaScript environment. If PAL only modifies values inside an isolated extension environment, page scripts will still see the original APIs.

Therefore, PAL uses a main-world injection strategy. Conceptually:

- The extension prepares hook scripts containing interceptors and perturbation logic.
- These scripts are injected into the page context early enough to preempt measurement scripts.

4.4.2 Early Hooking Requirement

Fingerprinting scripts can run at page start, often before the page is fully interactive. Therefore, PAL's injection strategy is designed to run as early as possible:

- At document start or equivalent earliest injection stage.
- Before common analytics libraries and tag managers execute.

This is crucial for correctness because missing early calls would produce inconsistent behavior and reduce drift effectiveness.

4.5 Context Propagation Implementation

4.5.1 Iframe Propagation

Iframes can be created dynamically. PAL's iframe propagation strategy ensures that when a frame is created:

- hook scripts are injected into the iframe's main world,
- the same policy (mode, persona seed, epoch) is applied,
- telemetry is labelled as iframe context for later analysis.

Iframe propagation is essential because many trackers run inside third-party frames to isolate themselves from top-level policies and restrictions.

4.5.2 Worker Propagation

Workers represent a more constrained environment. Worker scripts can execute fingerprint computations in background threads. PAL's worker propagation strategy aims to:

- ensure that workers receive the same epoch and persona seed inputs,
- apply hooks to worker-visible surfaces where feasible,
- log worker-specific measurements and timing.

Workers are also where implementation gaps are easiest to measure, because the evaluation explicitly reports drift and linkability in worker context. The current results show that certain surfaces (notably Audio drift) are not yet fully drifted in worker context, which is treated as an implementation gap rather than a conceptual flaw.

4.6 Deterministic Seed Derivation in Implementation

The conceptual seed formula described in the previous chapter is implemented in a concrete sequence:

1. Load or generate persona seed P.
2. Determine active epoch e.
3. For each surface x and context c, compute derived seed $S_{x,c,e}$.
4. Initialize a deterministic PRNG from $S_{x,c,e}$.
5. Generate perturbation parameters $\theta_{x,c,e}$.

For correctness, PAL must ensure that:

- P remains stable across the intended lifetime of the persona.
- e is consistent across contexts during a given measurement window.
- $S_{x,c,e}$ is computed identically in all contexts where the surface is hooked.

4.7 Surface Hooking and Perturbation Logic

This section describes the implementation pattern for each major surface. The goal is not to list every hook line-by-line, but to document the technique in a way that makes the system auditable.

4.7.1 Canvas Implementation

Canvas fingerprinting generally relies on reading pixel output. PAL's implementation therefore targets readout points rather than draw calls, because:

- draw calls are numerous and are used by legitimate graphics workloads,
- readout calls are more directly connected to fingerprint extraction.

Implementation pattern:

1. Identify canvas readout functions used in fingerprinting.
2. Wrap these functions with interceptors.
3. In privacy mode, apply deterministic perturbation using $\theta_{\text{Canvas},c,e}$.
4. Preserve return type and shape.

Correctness constraints:

- Output dimensions must remain unchanged.
- Pixel buffers must remain valid.
- Perturbations must be bounded to avoid visible degradation.

4.7.2 WebGL Implementation

WebGL fingerprinting includes both parameter queries and render-based hashes. PAL's WebGL implementation therefore has two conceptual hook categories:

- **Parameter hooks:** intercept queries that return identifying strings or parameters.
- **Readback hooks:** intercept readback operations used to extract rendering output as arrays.

Implementation pattern:

1. Wrap parameter query functions.
2. Optionally normalize or persona-align returned strings.
3. In privacy mode, apply bounded deterministic shifts to targeted parameters.
4. Wrap render readback functions and apply deterministic drift to extracted buffers where appropriate.

Correctness constraints:

- Returned strings must remain plausible.
- Numeric parameters must remain within valid ranges.
- WebGL contexts must continue functioning for real rendering workloads.

4.7.3 Audio Implementation

Audio fingerprinting often uses offline audio contexts and array extraction. PAL's audio implementation targets the derived array output or derived hash input so that:

- within-epoch output remains stable,
- across-epoch output shifts in privacy mode.

Implementation pattern:

1. Identify audio nodes and analysis pipeline points used in fingerprinting.
2. Intercept array extraction or final output readout.
3. Apply deterministic bounded perturbation based on $\theta_{\text{Audio},c,e}$ in privacy mode.

A critical implementation note is that audio contexts can behave differently across top, iframe, and worker environments. Therefore, PAL treats audio as a context-sensitive surface and explicitly measures drift outcomes by context.

4.7.4 Navigator and Screen Surfaces

Navigator and Screen surfaces are often used as supporting signals rather than the primary entropy source. PAL's implementation treats these surfaces carefully:

- In compatibility mode, persona coherence is enforced to avoid contradictions.
- In privacy mode, these surfaces may remain stable to preserve plausibility and reduce breakage risk.

This design choice aligns with the principle of applying drift primarily to high-entropy surfaces.

4.8 Telemetry Collection and Logging

PAL's evaluation pipeline relies on structured telemetry. The implementation records:

- surface outputs by site, epoch, mode, and context,
- error events and unhandled rejection events where captured,
- timing measurements for probe execution and hook overhead.

Each event is labelled with:

- run identifier,
- site identifier,
- mode,
- epoch,
- context,
- surface label.

The log format is JSONL to support streaming analysis and easy aggregation.

4.9 Robustness Handling in the Implementation

Because PAL operates by intercepting fundamental APIs, robustness is a major practical concern. PAL's implementation includes safeguards:

- try/catch around interception boundaries to avoid breaking page execution,
- fallback to original behavior when a hook fails,
- strict type preservation to avoid API contract violations,
- optional disable switches for surfaces to aid debugging.

These safeguards are important in ensuring that the system remains usable even while being evaluated and improved.

4.10 Summary

This chapter described PAL's implementation as a layered, deployable system. PAL centralizes mode routing and policy decisions, injects hook logic into the main world for page-visible APIs, propagates behavior across top, iframe, and worker contexts, and implements deterministic epoch-seeded perturbations for high-entropy surfaces. The telemetry pipeline collects structured measurements required to compute drift and linkability metrics. The implementation emphasizes correctness through bounded perturbations, API contract preservation, and robustness safeguards. The next chapter will describe the experimental protocol in detail, including the crawl configuration, measurement procedure, logging schema, and reproducibility steps.

CHAPTER 5

SYSTEM ARCHITECTURE

5.1 Architecture Overview

PAL is implemented as a deployable browser-extension based system that enforces two privacy-relevant behaviors at runtime:

- **Persona enforcement:** present a coherent and internally consistent browser persona to JavaScript-accessible surfaces.
- **Epoch-based drift (privacy mode):** introduce deterministic, bounded perturbations on selected high-entropy fingerprint surfaces so that composite fingerprints become less reusable across time.

To accomplish this under modern browser constraints, PAL is structured as a pipeline with two major planes:

- **Control plane:** configuration, mode selection, persona seed generation, epoch scheduling, and policy decisions.
- **Data plane:** runtime hooks, surface perturbation logic, context propagation (top/iframe/worker), and telemetry emission.

The architecture is designed to satisfy the invariants defined earlier, especially intra-epoch stability, inter-epoch drift for selected surfaces, mode separation, and cross-context coverage.

5.2 Browser Constraint Model

PAL is explicitly built under real-world extension constraints. These constraints strongly shape the architecture:

- **Isolated worlds:** extension scripts execute in an isolated environment, while page scripts execute in the main world. Modifying page-visible APIs requires main-world injection.
- **MV3 service worker model:** background logic is event-driven rather than persistent. Control signals and state must be designed to survive lifecycle churn.
- **Multi-context execution:** measurement and fingerprinting can occur in iframes and workers, not only the top frame.

Therefore, PAL uses a layered injection and propagation strategy to ensure that policy decisions (control plane) are correctly applied at the hook points (data plane) inside each context.

5.3 Component Breakdown

PAL is composed of the following core components.

5.3.1 C1: Configuration and Mode Manager

The configuration system governs:

- Active mode: vanilla, compatibility, or privacy.
- Enabled surfaces: which APIs are hooked and which are left unchanged.
- Drift policy: which surfaces drift, how drift is seeded, and how epochs are defined.

This component defines the active policy and ensures mode separation. In particular:

- **Vanilla:** policy is effectively "no interception" (baseline behavior).
- **Compatibility:** interception may occur, but drift is disabled, preserving stability.
- **Privacy:** interception occurs and epoch-based drift is enabled for selected surfaces.

5.3.2 C2: Persona Seed and Epoch Scheduler

PAL introduces an epoch abstraction. The epoch scheduler decides the current epoch value e and exposes it to all contexts. In privacy mode, e participates in seed derivation so that outputs remain stable inside the epoch but differ across epochs.

The persona seed is a stable internal secret used to derive deterministic perturbations. Conceptually, PAL uses the following approach:

- Generate or load a base persona seed P.
- Compute an epoch value e (either time-based or run-based).
- For each surface x, compute a surface seed S_x derived from (P, e, x, c), where c is the context.

This design supports controlled drift while maintaining intra-epoch stability.

5.3.3 C3: Main-World Injection Layer

Because page-visible JavaScript APIs must be modified in the main world, PAL includes an injection layer that:

- Deploys hook code into the top document.
- Ensures the same logic can be applied inside iframes.
- Provides worker support where possible through worker instrumentation or bootstrap logic.

This injection layer bridges extension logic to the runtime environment where tracking scripts execute.

5.3.4 C4: Hook Manager and Surface Interceptors

The hook manager implements the data plane. It defines hook points for relevant surfaces and routes them through surface-specific perturbation modules.

At a high level, for a given API call:

1. Intercept the call at the hook point.
2. Determine active mode and policy.
3. If compat mode: return coherent persona-stable output (no drift).
4. If privacy mode: apply deterministic epoch-seeded perturbation to the output.
5. Return the modified output while preserving the original API contract.

This component is the primary contributor to privacy effectiveness because it controls the observable outputs.

5.3.5 C5: Context Propagation Layer

Context propagation ensures policy is applied consistently in:

- top frame
- iframes
- workers

This is required because fingerprinting scripts frequently run in embedded or background contexts. PAL's evaluation also explicitly measures all three contexts, so propagation coverage is not optional, it is a measurable requirement.

5.3.6 C6: Telemetry and Evaluation Pipeline

PAL includes a research pipeline that collects measurement outputs and produces:

- JSONL event streams
- surface vectors by site, epoch, mode, and context
- derived metrics: drift, cross-context consistency, linkability, error signal, and timing overhead

This subsystem is critical because it converts PAL from "just an extension" into an empirically evaluated privacy system with measurable claims.

5.4 Control Plane vs Data Plane

PAL's architecture is easiest to understand by separating control plane and data plane responsibilities.

5.4.1 Control Plane Responsibilities

The control plane provides:

- A single source of truth for: mode, persona configuration, drift policy, and epoch value.
- Deterministic seed derivation inputs.
- Stable configuration dissemination across contexts.

Control plane decisions must be consistent across contexts. If two contexts disagree about epoch or policy, drift can become inconsistent and reduce the strength of the privacy guarantees.

5.4.2 Data Plane Responsibilities

The data plane performs:

- Runtime interception of fingerprint-relevant APIs.
- Surface-specific perturbation and output shaping.
- Context-level execution and propagation.
- Measurement logging and debug instrumentation.

The data plane must enforce invariants: stability within epoch, bounded perturbations, and correct mode separation.

5.5 Message Flow and Execution Sequence

This section describes the operational sequence of PAL in a typical browsing session. The flow differs slightly by context.

5.5.1 Top Frame Flow

The top frame is the first entry point. The general sequence is:

1. The extension loads and activates according to its policy.
2. The control plane establishes mode m, persona seed P, and epoch e.
3. The main-world injection layer installs hooks before fingerprint scripts execute.
4. Hooked surfaces route calls through the surface interceptors.
5. Telemetry is emitted to the evaluation pipeline where enabled.

This flow is designed to capture the earliest possible hooking point so that scripts that fingerprint at page start are intercepted.

5.5.2 Iframe Flow

Iframes may be created dynamically after page load, and they can contain third-party tracking logic. Therefore, PAL needs a strategy to ensure hooks are also applied in frames.

The iframe flow is:

1. An iframe is created by the site.
2. The propagation layer detects the iframe context.
3. Hooks are injected into the iframe main world.
4. The iframe uses the same control plane policy (mode, persona seed, epoch) as the top frame.
5. Telemetry is collected separately for iframe context.

The design goal is for iframe behavior to reflect the same mode semantics as the top frame, especially in privacy mode where drift should apply uniformly.

5.5.3 Worker Flow

Workers represent a distinct environment. Worker-based fingerprinting can occur because:

- workers can compute hashes and fingerprints without blocking the UI,
- worker scripts can reduce visibility of profiling logic,
- worker-based computations can be used as a side channel.

The worker flow is:

1. A worker is created by the site.
2. PAL attempts to ensure the worker environment receives the correct policy and seed inputs.
3. Worker-visible surfaces are intercepted where supported.
4. Telemetry is collected in worker context for later analysis.

Worker support is also where context-specific gaps can occur, which is why the evaluation explicitly reports worker drift and worker linkability.

5.6 Architecture Diagrams

This report uses a set of diagrams that correspond directly to the architecture described above. Insert your diagrams here. If you do not have them yet, create them as simple block diagrams. Even simple diagrams are better than no diagrams because they make the system traceable.

5.6.1 System Overview Diagram

fig_arch_system_overview.png

Figure 5.1: High-level PAL system overview showing control plane, data plane, and evaluation pipeline.

This figure should show the relationship between:

- the extension control logic (mode, persona, epoch),
- the injected hook layer (surface interceptors),
- contexts (top, iframe, worker),
- and the telemetry pipeline (JSONL logs and metrics).

5.6.2 Injection and Hooking Flow Diagram

fig_arch_injection_flow.png

Figure 5.2: Injection strategy and hook installation flow from extension world to main world across contexts.

This figure should trace:

- how policy is established,
- how hook code is installed,
- how calls are intercepted and routed through perturbation functions.

5.6.3 Epoch and Seed Derivation Diagram

fig_arch_epoch_seed.png

Figure 5.3: Conceptual seed derivation from persona seed, epoch, surface label, and context.

This diagram should visually encode the deterministic drift idea:

$$S_{x,c,e} = H(P \| e \| x \| c \| \text{optional keys})$$

and show how $S_{x,c,e}$ is used by each surface module to produce bounded perturbations.

5.7 How the Architecture Supports the Invariants

This section explicitly links architecture components to invariants.

5.7.1 Intra-Epoch Stability

Intra-epoch stability (Invariant I1) is achieved because:

- mode and epoch are stable inputs inside the epoch,
- surface perturbations are derived deterministically from those stable inputs,
- per-call randomness is avoided.

5.7.2 Inter-Epoch Drift in Privacy Mode

Inter-epoch drift (Invariant I2) is achieved because:

- the epoch value is part of the surface seed derivation,
- changing the epoch changes the derived seed, causing outputs to shift deterministically.

5.7.3 Mode Separation

Mode separation (Invariant I3) is enforced at the policy routing layer:

- vanilla bypasses hooks,
- compatibility applies coherence-preserving logic without epoch drift,
- privacy applies epoch drift on selected surfaces.

5.7.4 Context Robustness

Context robustness (Invariant I5) is achieved by propagation mechanisms that ensure:

- iframes receive the same policy and epoch inputs,
- workers receive policy inputs where supported,
- telemetry distinguishes context so gaps are measurable.

5.7.5 Bounded Perturbation

Bounded perturbation (Invariant I6) is enforced within each surface module. Each module must:

- preserve output type and shape,
- preserve API contracts,
- keep perturbations within plausible ranges,
- avoid invalid or extreme outputs.

5.8 Summary

This chapter described PAL's architecture as a deployable system under real extension constraints. The design separates control plane policy decisions from data plane runtime interception and perturbation. It treats execution context as first-class and propagates persona and epoch policy across top frames, iframes, and workers. The telemetry pipeline converts runtime behavior into measurable research metrics. The next chapter will describe the deterministic drift mechanism in detail, including the construction of seeds, surface-specific perturbation strategies, and how bounded drift is achieved while preserving coherence and stability.

CHAPTER 6

EXPERIMENTAL SETUP AND METHODOLOGY

6.1 Purpose of Evaluation

PAL is a defensive system that changes browser exposed behavior. Therefore, evaluation must answer two questions at the same time:

- **Does PAL reduce linkability?** This is captured by measuring whether fingerprint vectors remain stable across time and across contexts, and whether they drift when drift is intended.
- **What is the compatibility cost?** This is captured by measuring observable breakage signals while browsing real websites under different operational modes.

A strong evaluation requires repeatability. PAL therefore uses a controlled experiment design with fixed modes and epochs, a standardized fingerprint probe, and a structured telemetry format that can be analyzed offline.

6.2 Experimental Design

6.2.1 Independent Variables

The evaluation varies two primary independent variables:

1. **Mode:**
   - **Vanilla:** no PAL modifications, baseline browser behavior.
   - **Compatibility:** persona enforcement without epoch drift.
   - **Privacy:** persona enforcement with deterministic epoch based drift.
2. **Epoch:** a discrete identifier that changes between repeated runs of the same site in privacy mode. Within a single epoch, PAL should behave consistently.

A single experiment run consists of visiting a site under a selected mode and epoch, allowing scripts to execute, and running the probe that extracts fingerprint vectors and emits telemetry.

6.2.2 Dependent Variables

The evaluation measures three dependent categories:

- **Fingerprint vectors:** hash based identifiers derived from major fingerprint surfaces.
- **Cross context consistency:** agreement of vectors between top frame, iframe, and worker.
- **Breakage signals:** runtime errors, unhandled promise rejections, and CSP violation events.

6.2.3 Experimental Hypotheses

The evaluation is structured around clear hypotheses:

1. **H1 (Vanilla stability):** in vanilla mode, major fingerprint vectors remain stable across epochs for most sites.
2. **H2 (Compat stability):** in compatibility mode, vectors remain stable across epochs because the persona is stable, and cross context consistency improves compared to vanilla by reducing contradictions.
3. **H3 (Privacy drift):** in privacy mode, key vectors drift across epochs while staying stable within the same epoch.
4. **H4 (Breakage tradeoff):** privacy mode increases breakage signals relative to compatibility mode due to more aggressive modifications.

These hypotheses are not theoretical claims. They are directly testable through the collected data.

6.3 Website Corpus

PAL is evaluated on a corpus of websites. Each site is treated as a separate experimental unit. Sites are visited repeatedly across modes and epochs. The corpus should include a mix of:

- content heavy sites (news, blogs)
- script heavy sites (large web apps)
- sites with embedded third party resources
- sites known to use fingerprint defenses or bot detection

The goal is not to pick only easy targets, but to measure behavior on realistic traffic patterns.

6.4 Automation Framework

6.4.1 Crawler Responsibilities

PAL includes an automated crawler that orchestrates the experiment. For each site, the crawler:

1. Launches a controlled browser instance.
2. Sets the mode and epoch configuration for the run.
3. Navigates to the target URL.
4. Waits for the page to stabilize and scripts to execute.
5. Executes the fingerprint probe in the page.

6. Collects structured telemetry emitted by the page.
7. Writes telemetry to a JSONL file for offline analysis.

This automation ensures the same measurement procedure is applied across the entire corpus.

6.4.2 Document Start Configuration

To ensure that configuration is applied before site scripts execute, the crawler injects a configuration object at `document_start`. This configuration includes:

- `run_id`
- `mode`
- `epoch_id`
- `site_key` or top level site identifier

This ensures PAL hook logic can behave deterministically during experiments and that telemetry events can be linked to the correct run context.

6.5 Fingerprint Probe Design

6.5.1 Probe Goals

The fingerprint probe is designed to:

- extract representative signals from multiple fingerprint surfaces
- run in multiple execution contexts (top, iframe, worker)
- produce a comparable vector format across sites
- avoid heavy dependence on site specific DOM structure

The probe focuses on surfaces that are both widely used and measurable using lightweight scripts.

6.5.2 Contexts Measured

The probe runs in three contexts:

1. **Top frame:** the main window environment.
2. **Iframe:** an injected subframe, typically using a controlled `about:blank` or blob based iframe.
3. **Worker:** a dedicated worker created using a blob script that runs the same extraction logic.

This design ensures that the same fingerprint extraction logic is applied across contexts, enabling direct consistency comparisons.

6.5.3 Surfaces Extracted

The probe extracts from the following surfaces:

- **Canvas:** obtain a pixel buffer and compute a hash.
- **WebGL:** read pixel output or key parameters and compute a hash.
- **Audio:** extract a sample array and compute a hash.
- **Navigator:** capture a set of properties and hash them.
- **Screen:** capture screen attributes and hash them.

Each surface output is converted into a compact identifier using a stable hash function. The final fingerprint vector is represented as a structured record rather than a single concatenated string. This improves interpretability and allows surface level analysis.

6.6 Telemetry Collection

6.6.1 Event Format

Telemetry is emitted as structured events from the page. Events include:

- mode, epoch, run identifier
- URL and site key
- context identifier (top, iframe, worker)
- surface specific hashes
- breakage signals

Events are emitted through a channel that the crawler can capture reliably, such as console logging with a recognizable prefix. This design avoids dependence on network logging or privileged browser APIs.

6.6.2 JSONL Storage

Events are stored as JSON Lines (JSONL). JSONL provides:

- line by line append behavior, robust to partial failures
- easy streaming and filtering
- compatibility with common data processing scripts

Each line represents one event. A single run can produce multiple events, for example one fingerprint vector per context plus additional breakage signals.

6.7 Metrics

6.7.1 Intra Epoch Stability

Intra epoch stability measures whether repeated calls within the same epoch produce consistent fingerprints. For each site and context, a surface is stable within an epoch if the hash does not change across repeated measurements.

This metric validates that PAL does not introduce uncontrolled randomness that could break applications and also validates the deterministic design of privacy mode.

6.7.2 Inter Epoch Drift

Inter epoch drift measures whether fingerprints change when the epoch changes. In privacy mode, drift is desired. In compatibility mode, drift is not desired.

For each site, context, and surface:

- drift is recorded if the hash differs across epochs
- drift magnitude can be summarized as the fraction of surfaces that change

A strong privacy mode result is one where high value surfaces drift across epochs while remaining stable within epoch.

6.7.3 Cross Context Consistency

Cross context consistency measures whether the same site produces coherent identity outputs in top, iframe, and worker contexts.

For each surface:

- consistency is recorded if the hash agrees between contexts
- inconsistency is recorded if the hash differs, which may indicate a propagation gap or a mismatch in enforcement

This metric is essential because a tracker can fingerprint from multiple contexts and use mismatches as signals.

6.7.4 Breakage Rate

Breakage is approximated through runtime signals:

- page errors
- unhandled promise rejections
- CSP violations

For each site and mode, the breakage rate can be summarized as:

- whether any breakage signal occurred
- count of breakage signals per visit
- distribution of error types if classified further

This metric does not capture all forms of functional breakage, but it provides an empirical indicator that can be collected at scale.

6.8 Controls and Validity Considerations

6.8.1 Control Conditions

Vanilla mode acts as a baseline for understanding natural stability. Compatibility mode acts as a stable persona baseline for measuring the effect of enforcement without drift. Privacy mode tests the drift mechanism.

This three condition setup allows PAL to isolate the contribution of persona enforcement and the contribution of epoch drift.

6.8.2 Noise Sources and Mitigation

Experiments can be affected by:

- dynamic page content that changes rendering outputs
- asynchronous loading differences across visits
- nondeterminism in GPU and audio pipelines

PAL mitigates these issues by:

- using a probe that draws controlled content before extraction
- hashing results to produce compact comparable identifiers
- repeating measurements across epochs and contexts

6.8.3 Ethical and Practical Considerations

The crawler is designed to behave as a passive measurement tool. It loads pages, runs probes, and records outputs. It does not attempt to bypass paywalls, perform account logins, or interact with user data. The evaluation focuses on measuring PAL's behavior and not on exploiting websites.

6.9 Summary

This chapter described the experimental methodology used to evaluate PAL, including the mode and epoch design, crawler automation, fingerprint probe structure, telemetry collection, and the metrics used to quantify stability, drift, cross context consistency, and breakage signals. The next chapter presents results and analysis based on the collected measurements.

CHAPTER 7

EXPERIMENTAL PROTOCOL AND REPRODUCIBILITY

7.1 Why a Formal Protocol is Required

A browser-privacy defense is only credible if it can be evaluated in a way that is:

- **repeatable:** the same procedure yields the same interpretation and comparable numbers,
- **traceable:** results can be connected to raw measurements and logging artifacts,
- **context-aware:** evaluation includes realistic execution contexts beyond the top frame,
- **mode-comparable:** vanilla, compatibility, and privacy modes can be compared under identical conditions.

This chapter defines the protocol used to generate the measurement dataset and the derived metrics presented later. The emphasis is not on ideal laboratory conditions, but on a realistic web-crawl evaluation that reflects how fingerprinting scripts execute on real sites.

7.2 Evaluation Setup Summary

The evaluation performed for this report used the following setup:

- **Sites:** 19 websites.
- **Modes:** vanilla, compatibility, privacy.
- **Epochs:** 3.
- **Execution contexts:** top frame, iframe, worker.
- **Total measurement vectors:** 1015.

These values determine the scale of the dataset used in the metrics and are sufficient to demonstrate clear differences between modes, especially for high-entropy drifted surfaces and composite linkability.

7.3 Independent Variables

PAL's evaluation is structured around the following independent variables:

7.3.1 Mode (m)

The primary independent variable is the operational mode:

$$m \in \{\text{vanilla}, \text{compat}, \text{privacy}\}$$

7.3.2 Epoch (e)

Epoch is the time-like variable used to measure cross-epoch stability and drift:

$$e \in \{1, 2, 3\}$$

7.3.3 Context (c)

Context models where the measurement script executes:

$$c \in \{\text{top}, \text{iframe}, \text{worker}\}$$

7.3.4 Site (s)

Each measurement is associated with a site $s \in \mathcal{S}$, where $|\mathcal{S}| = 19$.

7.4 Dependent Variables

The evaluation measures the following dependent variables:

- Surface outputs $V_{s,e,c,m}(x)$ for each surface x.
- Composite fingerprint keys $K_{s,e,c,m}$.
- Derived metrics: drift rate, cross-context consistency, composite linkability.
- Deployability signals: error events and timing overhead.

7.5 Crawl Procedure

This section defines the procedure used for each crawl run.

7.5.1 Step 1: Select Mode and Initialize Policy

For each mode m:

1. PAL is configured to run in mode m.
2. The persona seed and policy settings are initialized.
3. The epoch scheduler is set to enable the planned number of epochs.

7.5.2 Step 2: Iterate Epochs

For each epoch $e \in \{1, 2, 3\}$:

1. The epoch identifier is set or advanced.
2. A site list crawl is executed under the same epoch.
3. Measurements are collected in top, iframe, and worker contexts.

The purpose of holding e stable during each epoch crawl is to ensure that PAL's intra-epoch stability requirement is evaluated correctly.

7.5.3 Step 3: Site Visit and Context Probing

For each site s:

1. Load the site in a controlled browser session.
2. Execute the probe script in the **top frame**.
3. Execute the probe script in an **iframe** context.
4. Execute the probe script in a **worker** context.
5. Log outputs and timing for each context.

The iframe and worker probes are important because a significant fraction of fingerprinting scripts can run in embedded or background contexts.

7.5.4 Step 4: Event Capture

During site visits, PAL records:

- runtime error events (page-level JS errors),
- unhandled promise rejections (where enabled),
- CSP violation events (where enabled).

These are treated as coarse indicators of interference or compatibility pressure, not as strict measures of user-visible breakage.

7.6 Logging and Dataset Construction

7.6.1 JSONL Logging Format

PAL writes logs in JSONL (JSON Lines) format. Each line corresponds to one event or measurement record. JSONL is suitable because:

- it supports streaming and incremental parsing,
- it allows large-scale crawl output to be analyzed without loading everything into memory,
- it preserves per-event granularity for debugging.

7.6.2 Required Fields for Reproducibility

Each measurement record must include the following required fields:

- **run_id:** unique identifier of the crawl run.
- **site_id or url:** the site being measured.
- **mode:** vanilla, compat, or privacy.
- **epoch:** integer epoch identifier.
- **context:** top, iframe, or worker.
- **surface outputs:** fields representing each surface's measurement output (raw or hashed).
- **timestamp:** measurement time (optional but recommended).

Separately, error events should include:

- event type (error, rejection, CSP),
- message,
- stack trace or source URL where available,
- site identifier,
- mode and context.

Timing events should include:

- probe duration in milliseconds,
- mode and context,
- site identifier,
- epoch.

7.6.3 Dataset Aggregation

Once the crawl completes, the JSONL logs are aggregated into a measurement dataset. The aggregation performs:

- grouping by (site, mode, epoch, context),
- extraction of per-surface outputs $V_{s,e,c,m}(x)$,
- construction of composite keys $K_{s,e,c,m}$,
- computation of drift, consistency, and linkability metrics.

7.7 Metric Computation Procedure

This section summarizes how the formal metrics are computed from logs.

7.7.1 Inter-Epoch Drift

For each surface x:

1. For each (site, context, mode), compare outputs across epoch pairs.
2. Record whether values differ.
3. Aggregate across sites and epoch pairs to compute drift rate.

7.7.2 Cross-Context Consistency

For each surface x:

1. For each (site, epoch, mode), check whether top, iframe, and worker outputs match.
2. Aggregate matches across sites and epochs to compute a consistency rate.

7.7.3 Composite Linkability

For the composite key K:

1. For each (site, context, mode), compare K across epoch pairs.
2. Record whether the composite key matches exactly.
3. Aggregate across sites and epoch pairs to compute linkability.

7.7.4 Error Events

For each mode:

1. Count total error events.
2. Count number of sites with at least one error event.

Errors are not treated as proof of breakage, but they provide an engineering signal.

7.7.5 Timing Overhead

For each (context, mode):

1. Extract timing samples from the logs.
2. Compute mean and standard deviation.

Timing is used to quantify instrumentation cost and highlight optimization targets.

7.8 Reproducibility Checklist

A reproducible evaluation must specify:

- Browser version and platform (OS, CPU architecture).
- Extension version or Git commit hash.
- Configuration file values (mode settings, enabled surfaces, epoch definition).
- Site list used for crawling.
- Number of epochs and context probing strategy.
- Output log filenames and analysis scripts used to compute metrics and graphs.

This report provides the final aggregated metrics and figures, and the logging design ensures that any metric can be traced back to raw JSONL event lines.

7.9 Threats to Validity

This evaluation is realistic but not exhaustive. The main validity threats are:

7.9.1 Limited Site Set

The site set contains 19 sites. While sufficient to demonstrate clear mode separation and strong drift on targeted surfaces, a larger crawl would provide tighter confidence intervals and better coverage of edge cases.

7.9.2 Site Variability

Sites vary in their runtime behavior, use of third-party scripts, and error baseline. Error event counts therefore reflect both PAL-induced effects and normal site noise.

7.9.3 Browser and Hardware Dependence

Fingerprint surfaces can depend on the underlying hardware and browser version. Reproducing results across different machines may yield different baseline fingerprints, but PAL's mode differences and drift behavior should remain observable.

7.10 Summary

This chapter defined the experimental protocol used to evaluate PAL. The evaluation crawled 19 sites across three modes, three epochs, and three execution contexts, yielding 1015 measurement vectors. It described the crawl procedure, logging format, dataset aggregation process, and metric computation rules. It also provided a reproducibility checklist and identified threats to validity. The next chapter presents the results

and analysis, interpreting drift, cross-context consistency, composite linkability, error signals, and timing overhead in the context of the attacker model and design invariants.

CHAPTER 8

RESULTS AND ANALYSIS

8.1 How to Read This Chapter

This chapter reports the empirical outcomes produced by PAL under the protocol defined previously. The results are organized to directly reflect the attacker model and the system invariants:

- **Drift** measures whether targeted surfaces change across epochs in privacy mode while staying stable in vanilla and compatibility modes.
- **Cross-context behavior** indicates whether PAL's policy is applied consistently across top, iframe, and worker contexts.
- **Composite linkability** is the most important metric because it approximates the attacker's strict-equality success condition for reusing a fingerprint across time.
- **Error and timing** quantify deployability pressure and identify concrete engineering targets without re-defining the core privacy objective.

The evaluation dataset consists of:

- 19 sites
- 3 modes (vanilla, compat, privacy)
- 3 epochs
- 3 contexts (top, iframe, worker)
- 1015 total measurement vectors

Unless otherwise stated, all rates are computed as defined in the formal metrics chapter.

8.2 Inter-Epoch Drift Rates

8.2.1 Canvas Drift

Canvas is one of the highest leverage fingerprint surfaces. PAL targets it explicitly in privacy mode through deterministic epoch-based perturbation on pixel readout.

Observed drift rates:

- **Top:** Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 1.00$
- **Iframe:** Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 0.73$

- **Worker:** Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 0.93$

Interpretation:

- In top context, Canvas achieves full drift in privacy mode, which strongly supports the claim that PAL can disrupt a major fingerprint component across epochs.
- Worker context also shows strong drift in privacy mode, indicating that PAL's core deterministic drift mechanism is not limited to only top-frame execution.
- Iframe context shows partial drift rather than full drift. This is still a meaningful improvement over vanilla and compat, but it identifies a concrete propagation gap: in some cases, iframe execution does not receive fully consistent hook behavior or the probing environment differs in a way that reduces drift activation.

Insert your Canvas drift visualization here:

fig_results_canvas_drift.png

Figure 8.1: Canvas inter-epoch drift rates by context for vanilla, compatibility, and privacy modes.

8.2.2 WebGL Drift

WebGL is another high-entropy surface with strong fingerprinting leverage. PAL targets WebGL in privacy mode using deterministic perturbations on selected query outputs and/or render readback behavior.

Observed drift rates:

- **Top:** Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 1.00$
- **Iframe:** Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 0.72$
- **Worker:** Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 0.93$

Interpretation:

- WebGL drift matches the intended behavior pattern: stable in vanilla and compat, strongly drifting in privacy mode.
- The same context pattern appears as with Canvas: top and worker contexts show strong drift, iframe shows partial drift. This provides evidence that the deterministic drift mechanism is effective, and that the main remaining work is to improve uniform propagation and activation in iframe environments.

Insert your WebGL drift visualization here:

fig_results_webgl_drift.png

Figure 8.2: WebGL inter-epoch drift rates by context for vanilla, compatibility, and privacy modes.

8.2.3 Audio Drift

Audio fingerprinting can be highly identifying and is commonly used in combination with graphics surfaces. PAL attempts to apply deterministic drift to audio measurements in privacy mode.

Observed drift rates:

- **Top:** Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 1.00$
- **Iframe:** Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 0.62$
- **Worker:** Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 0.00$

Interpretation:

- In top context, audio drift achieves full drift in privacy mode, which shows that PAL's approach can disrupt audio-derived fingerprints when the hook path executes as intended.
- In iframe context, drift is partial. This again suggests that iframe environments are an area where propagation and activation require further refinement.
- In worker context, audio drift is currently not observed. This is a known gap and a highly valuable engineering target because worker-based measurement is a realistic attacker strategy.

This is not framed as a weakness of the concept, but as evidence that the evaluation pipeline is successfully identifying where context-level implementation must be extended to fully close coverage gaps.

Insert your Audio drift visualization here:

fig_results_audio_drift.png

Figure 8.3: Audio inter-epoch drift rates by context for vanilla, compatibility, and privacy modes. Worker-audio drift is currently absent, motivating future work on worker audio instrumentation.

8.2.4 Other Surfaces

For the remaining surfaces measured (Navigator, Screen, Intl), observed drift rates are:

- Navigator: all contexts show 0.00 drift in all modes.
- Screen: all contexts show 0.00 drift in all modes.
- Intl: all contexts show 0.00 drift in all modes.

Interpretation:

- These surfaces are treated primarily as coherence surfaces rather than drift surfaces in the current design.

- Keeping them stable helps preserve plausibility and avoids widespread breakage from changing environment-reported values.
- The major privacy impact is achieved through drift on high-entropy surfaces (Canvas, WebGL, and Audio), which is consistent with PAL's design principle of focusing drift where it yields the most reduction in composite linkability.

8.3 Cross-Context Consistency

Cross-context consistency measures whether a surface matches across top/iframe/worker contexts within the same epoch. This metric is relevant for both detectability and for understanding how uniform the policy application is.

Observed consistency rates:

8.3.1 Canvas Consistency

- Vanilla $= 1.00$
- Compat $= 1.00$
- Privacy $= 0.00$

Interpretation:

- In vanilla and compatibility modes, Canvas outputs match across contexts, as expected under stability semantics.
- In privacy mode, cross-context Canvas consistency is 0.00. This indicates that while drift is effective, it is not producing identical values across contexts within the same epoch. This is not necessarily undesirable, because a defense may deliberately allow context-specific variation. However, it does imply that cross-context coherence is not currently enforced for these surfaces in privacy mode.

8.3.2 WebGL Consistency

- Vanilla $= 1.00$
- Compat $= 1.00$
- Privacy $= 0.00$

Interpretation mirrors Canvas: in privacy mode, WebGL is not consistent across contexts within an epoch, reflecting context-specific behavior and/or differences in hook activation across contexts.

8.3.3 Navigator Consistency

- Vanilla $= 1.00$
- Compat $= 1.00$

- Privacy $= 1.00$

Interpretation:

- Navigator fields remain coherent across contexts in all modes, which supports persona coherence and reduces trivial inconsistency signals.

8.3.4 Audio and Screen Consistency

- Audio: Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 0.00$
- Screen: Vanilla $= 0.00$, Compat $= 0.00$, Privacy $= 0.00$

Interpretation:

- For these surfaces, the measured consistency rate suggests that outputs vary across contexts even in vanilla. This may arise from how the probe is constructed or from differences in how these surfaces behave across contexts by default.
- Because consistency is not high even in vanilla, this metric is less informative for these surfaces without deeper per-surface analysis and may be refined in future evaluation iterations.

8.4 Composite Linkability

Composite linkability is the central outcome because it reflects the attacker's strict-equality ability to reuse the composite fingerprint key across epochs.

Observed composite linkability rates:

8.4.1 Top Context

- Vanilla $= 1.00$
- Compat $= 1.00$
- Privacy $= 0.00$

Interpretation:

- In vanilla and compatibility modes, the composite fingerprint is fully reusable across epochs, which matches the expected baseline.
- In privacy mode, top-context linkability is reduced to 0.00. This is the strongest possible outcome under strict-equality measurement and provides direct evidence that PAL achieves its primary objective in the most common context.

8.4.2 Iframe Context

- Vanilla $= 1.00$
- Compat $= 1.00$
- Privacy $= 0.27$

Interpretation:

- In privacy mode, iframe linkability decreases from 1.00 to 0.27.
- This shows meaningful disruption: most composite keys are no longer identical across epochs.
- The remaining 0.27 indicates that in some iframe cases, the composite fingerprint is still reusable. This is consistent with the partial drift rates observed in iframe for Canvas, WebGL, and Audio.

8.4.3 Worker Context

- Vanilla $= 1.00$
- Compat $= 1.00$
- Privacy $= 0.07$

Interpretation:

- In privacy mode, worker linkability drops sharply to 0.07.
- This is a strong result because it shows that PAL is not purely top-frame limited. Even with the observed worker-audio drift gap, the composite key rarely remains identical across epochs in worker context, likely due to strong Canvas and WebGL drift in worker.

Insert your composite linkability visualization here:

fig_results_composite_linkability.png

Figure 8.4: Composite fingerprint linkability across epochs by context and mode. Privacy mode eliminates strict-equality linkability in the top frame and sharply reduces it in iframe and worker contexts.

8.5 Error Signals and Interpretation

Runtime error events were captured during the crawl. Errors are a noisy signal because modern websites often emit baseline errors even without intervention. Therefore, PAL interprets errors as an **engineering robustness signal** rather than a direct measure of user-visible breakage.

Observed error event counts:

- **Vanilla:** 42 total error events; 12 / 19 sites with any error
- **Compat:** 73 total error events; 15 / 19 sites with any error
- **Privacy:** 72 total error events; 16 / 19 sites with any error

Interpretation:

- Vanilla already shows non-trivial errors, confirming that the web has baseline instability and that error counts cannot be treated as breakage by default.
- Compatibility and privacy show higher error counts and a larger number of sites with at least one error event. This is expected pressure when intercepting or wrapping core APIs.
- Importantly, the difference between compat and privacy is small in aggregate error count, suggesting that the core hooking layer is the dominant contributor, rather than drift itself.

Insert your error visualization here:

fig_results_error_events.png

Figure 8.5: Total captured error events by mode. Vanilla also exhibits baseline errors, indicating that error counts should be interpreted as a robustness signal rather than a definitive breakage measure.

8.6 Timing Overhead and Performance Cost

Timing overhead measures the runtime cost of probing and instrumentation. This is a critical deployability factor because a privacy tool that causes extreme latency can harm user experience and increase detection risk.

Observed timing statistics (mean and standard deviation in milliseconds):

8.6.1 Top Context

- Vanilla: mean $= 24.8$, std $= 6.6$
- Compat: mean $= 177.5$, std $= 26.8$
- Privacy: mean $= 275.4$, std $= 168.5$

8.6.2 Iframe Context

- Vanilla: mean $= 28.1$, std $= 29.3$
- Compat: mean $= 493.6$, std $= 609.1$
- Privacy: mean $= 515.5$, std $= 582.2$

8.6.3 Worker Context

- Vanilla: mean $= 24.2$, std $= 10.2$
- Compat: mean $= 400.9$, std $= 734.6$
- Privacy: mean $= 450.6$, std $= 571.1$

Interpretation:

- Both compat and privacy introduce overhead relative to vanilla, reflecting the cost of interception and additional computation.
- Overhead is particularly high and highly variable in iframe and worker contexts, indicated by large standard deviations. This suggests that timing costs depend strongly on site conditions and that some sites trigger much heavier instrumentation paths than others.
- Privacy overhead is higher than compat, but the dominant cost appears to come from the general hook and measurement pipeline rather than drift alone, since the gap between compat and privacy is smaller than the gap between vanilla and compat.

Insert your timing visualization here:

fig_results_timing_overhead.png

Figure 8.6: Probe timing (mean and std) by context and mode. Compatibility and privacy modes introduce overhead relative to vanilla; variability is highest in iframe and worker contexts.

8.7 Key Findings

This section summarizes the most important outcomes in terms of the attacker's ability to reuse fingerprints.

8.7.1 Finding F1: Privacy Mode Eliminates Strict Linkability in the Top Frame

In top context, privacy mode reduces composite linkability from 1.00 to 0.00. Under the strict-equality attacker model, this means that the attacker cannot reuse the composite fingerprint across epochs in the top frame.

8.7.2 Finding F2: Privacy Mode Strongly Reduces Linkability in Iframes and Workers

Privacy mode reduces linkability to:

- 0.27 in iframe
- 0.07 in worker

These values demonstrate that PAL's defenses extend beyond the top frame and that strong drift in Canvas and WebGL contributes significantly to disrupting composite equality.

8.7.3 Finding F3: Worker Audio Drift is a Current Coverage Gap

Audio drift is currently absent in worker context. Despite this, worker linkability is still low due to strong drift in other high-entropy surfaces. Improving worker audio drift remains a clear next-step enhancement to further reduce residual linkability and to improve uniform coverage.

8.7.4 Finding F4: Overhead and Errors Identify Engineering Optimization Targets

Error counts increase in compat and privacy relative to vanilla, but vanilla already has baseline errors. Timing overhead is higher in compat and privacy and shows large variance in iframe and worker contexts. These outcomes motivate targeted optimization and robustness work without invalidating the core privacy outcome, which is the reduction of composite strict linkability across epochs.

8.8 Summary

This chapter presented the empirical results of PAL under a real-site crawl across 19 sites, three modes, three epochs, and three execution contexts. In privacy mode, PAL achieves strong inter-epoch drift on Canvas and WebGL in top and worker contexts and partial drift in iframe contexts. Audio drift is strong in top, partial in iframe, and absent in workers, indicating a context-specific instrumentation gap. Most importantly, composite linkability drops from 1.00 in vanilla and compatibility modes to 0.00 in top context, 0.27 in iframes, and 0.07 in workers, demonstrating that PAL can significantly reduce the reuse of strict-equality fingerprint keys across time. Error and timing signals highlight engineering targets for robustness and performance optimization, which are addressed in the next chapter on discussion and future work.

CHAPTER 9

THREAT MODEL

9.1 Purpose and Scope

This chapter defines the attacker model used to evaluate PAL. A precise threat model is necessary because browser privacy defenses can look strong or weak depending on what the attacker is assumed to do. PAL is designed around a realistic, web-deployed tracking adversary who uses JavaScript-accessible fingerprinting surfaces to produce a stable identifier and link a user across time.

The threat model here is written to match what PAL actually measures: surface outputs and strict-equality linkability across epochs, across multiple execution contexts.

9.2 Assets to Protect

PAL aims to protect the following assets:

- **Cross-time unlinkability:** reduce the probability that the same user can be reliably linked across epochs using a composite fingerprint.
- **Context robustness:** prevent an attacker from bypassing defenses by moving fingerprint collection into iframes or workers.
- **Plausible execution environment:** keep outputs within plausible bounds so that privacy behavior does not trivially identify itself.

PAL does not aim to provide full anonymity or eliminate all tracking. The goal is more specific: reduce strict-equality reuse of composite fingerprint keys across time under a realistic JavaScript attacker.

9.3 Adversary Capabilities

The attacker is modeled as a web-deployed script capable of running arbitrary JavaScript in the user's browser in one or more contexts. This includes:

9.3.1 C1: Script Execution Capability

The attacker can execute code:

- in the **top frame** of a visited site,
- in **iframes**, including third-party iframes embedded within a top site,
- in **web workers** spawned by the page.

This is a critical capability because modern tracking systems often use third-party frames and workers to isolate their logic and increase measurement reliability.

9.3.2 C2: Access to Fingerprinting APIs

The attacker can call and combine outputs from common fingerprint surfaces, including:

- Canvas readout APIs
- WebGL parameter queries and render readbacks
- Audio processing and analysis pipelines
- Navigator and Screen properties
- Locale and formatting signals (Intl family)

The attacker may also query surfaces repeatedly to test stability, detect modification, or extract higher-quality signatures.

9.3.3 C3: Cross-Site Presence

The attacker can be present across multiple sites through:

- third-party scripts loaded by many sites,
- tag managers and ad networks,
- embedded analytics libraries.

This capability matters because a fingerprint reused across time and across sites becomes a powerful global identifier.

9.3.4 C4: Cross-Time Observation

The attacker can observe the same user over time by re-collecting fingerprints on different days or different sessions. PAL's epoch abstraction is a proxy for this time dimension.

9.3.5 C5: Server-Side Storage and Correlation

The attacker can store collected fingerprints server-side and compare new observations against past observations.

9.3.6 C6: Adaptive Probing

The attacker can adapt probing behavior, for example:

- probe multiple contexts and choose the one that yields the most stable output,

- probe repeatedly and average, filter, or select stable outputs,
- apply feature detection to choose specific surfaces based on browser capabilities.

PAL's evaluation assumes the attacker can probe all three contexts, which is why context coverage and context-level linkability are measured.

9.4 Adversary Goals

The attacker's primary goal is:

- **G1: Persistent identification across time.** Given a current fingerprint observation, match it to the same user's past observations with high confidence.

Secondary goals include:

- **G2: Cross-site linking.** Link the same user across multiple sites based on stable fingerprint components.
- **G3: Defense detection.** Detect whether a privacy defense is active so that the attacker can segment users or apply specialized probing techniques.

PAL's evaluation directly addresses G1 through strict-equality linkability across epochs and partially addresses G3 through coherence constraints and bounded perturbations.

9.5 Attacker Success Criterion

PAL evaluates a strict attacker success criterion that is intentionally strong:

9.5.1 Strict-Equality Composite Key

Let the attacker compute a composite key:

$$K_{s,e,c} = \mathcal{F}\left(V_{s,e,c}(x_1), V_{s,e,c}(x_2), \ldots, V_{s,e,c}(x_n)\right)$$

where $x_1, \ldots, x_n$ are fingerprint surfaces and $\mathcal{F}$ is a deterministic composition function (for example concatenation followed by hashing).

The strict success criterion is:

$$K_{s,e,c} = K_{s,e',c} \quad \text{for } e \neq e'$$

If the composite key remains identical across epochs, the attacker can reuse the identifier with strict equality.

9.5.2 Why Strict Equality is a Reasonable Baseline

In practice, many tracking systems do not require perfect equality, they can use fuzzy matching or probabilistic models. However, strict equality remains a meaningful baseline because:

- many production fingerprinting pipelines use stable hashes that are expected to be repeatable,
- strict equality is the easiest correlation mechanism and often used when available,
- eliminating strict equality reuse is a necessary first step before evaluating resistance to fuzzy matching.

PAL therefore aims to break strict equality reuse as the core measurable objective.

9.6 Attacker Variants

To clarify scope, the threat model includes several attacker variants.

9.6.1 A1: First-Party Fingerprinter

A first-party attacker runs scripts under the top-level site origin. They can call APIs in top context and can create iframes and workers.

9.6.2 A2: Third-Party Embedded Fingerprinter

A third-party attacker is embedded across many sites through third-party scripts and frames. They may run primarily inside third-party iframes.

This attacker is important because it motivates iframe coverage and the evaluation of iframe linkability.

9.6.3 A3: Worker-Focused Fingerprinter

A worker-focused attacker moves probing into workers to:

- reduce UI thread interference,
- hide probing logic,
- avoid some frame-level restrictions.

This attacker motivates worker coverage and the reporting of worker drift and linkability.

9.6.4 A4: Detection-Oriented Fingerprinter

A detection attacker attempts to identify anti-fingerprinting defenses. Techniques include:

- repeated calls to test per-call randomness,
- cross-context comparisons to detect inconsistency,
- probing API property descriptors to detect wrapping,
- measuring timing artifacts in hooked paths.

PAL's design directly resists some detection strategies by avoiding per-call randomness and by preserving API contracts, but detection remains a realistic pressure and is discussed further in the security analysis chapter.

9.7 Out of Scope

This threat model intentionally excludes several attacker capabilities that are not targeted by PAL's current design.

9.7.1 O1: Network-Level Tracking

PAL does not prevent tracking based on IP address, TLS fingerprints, or network metadata.

9.7.2 O2: Account-Based Tracking

If a user logs into an account, the site can track them by identity regardless of fingerprint stability.

9.7.3 O3: Hardware-Level or OS-Level Side Channels

PAL does not address side channels such as power analysis, CPU microarchitectural leakage, or non-JS hardware probing beyond the targeted browser surfaces.

9.7.4 O4: Full Browser Compromise

If malware or a malicious extension has privileged access, PAL cannot protect against that attacker.

These exclusions are standard and allow PAL's evaluation to remain focused on a realistic and practical web fingerprinting adversary.

9.8 Mapping the Threat Model to PAL's Mechanisms

This section summarizes how PAL's design directly addresses the attacker model.

9.8.1 Against Strict Equality Reuse

PAL's deterministic drift ensures that, in privacy mode, selected high-entropy surfaces change across epochs. Therefore, the composite key K is unlikely to remain identical across epochs, breaking the strict success criterion.

9.8.2 Against Context Shifting

Because PAL evaluates and defends in top, iframe, and worker contexts, the attacker cannot trivially bypass defenses by collecting fingerprints only in iframes or workers. The results show strong reductions in composite linkability in those contexts, though some gaps remain (notably worker audio drift and partial iframe drift).

9.8.3 Against Simple Detection by Instability

Because drift is epoch-based rather than per-call, PAL avoids a common detection signature of naive randomization: repeated calls yielding different values inside the same session.

9.9 Summary

This chapter defined PAL's threat model as a realistic JavaScript fingerprinting adversary capable of collecting composite fingerprints across top frames, iframes, and workers, and correlating them across time. The attacker's strict success criterion is reuse of an identical composite key across epochs. PAL's primary objective is to disrupt this equality condition through deterministic drift of high-entropy surfaces in privacy mode while preserving intra-epoch stability. The model also identifies detection-oriented attackers and motivates future work on cross-context coherence, worker audio coverage, and performance optimizations. The next chapter provides a security analysis that evaluates detection risks, bypass strategies, and residual linkability under this threat model.

CHAPTER 10

SECURITY ANALYSIS AND ROBUSTNESS

10.1 What This Chapter Covers

This chapter analyzes PAL against the threat model, focusing on four practical questions:

- How might an attacker **detect** that PAL is active?
- How might an attacker **bypass** PAL by shifting surfaces or contexts?
- What is the **residual linkability** that still remains, and why?
- What robustness properties must hold so that PAL remains deployable in real browsing?

The goal is to treat PAL like a security system: identify the realistic attacks, explain which are handled already, and isolate the exact technical work required to reduce remaining risk.

10.2 Detection Analysis

A strong tracker will not only attempt to collect a fingerprint, but will also attempt to detect modifications and either (a) adapt probing strategy or (b) segment the user population. PAL therefore must minimize trivial detection signals.

10.2.1 D1: Detection via Per-Call Instability

Attack: Call the same fingerprint function repeatedly in a tight loop and check whether output changes across calls.

Why it works against naive noise: Per-call randomness causes a clear inconsistency signal.

PAL status: PAL resists this class of detection by enforcing deterministic behavior within an epoch. The drift occurs on epoch boundary rather than per-call. Therefore, repeated calls inside an epoch should not show instability.

Residual risk: If epoch boundaries are too short or are triggered unexpectedly during a single page session, an attacker may observe changes during a browsing session. This can be reduced by choosing epoch definitions that do not change within a typical page lifetime.

10.2.2 D2: Detection via Wrapped Function Signatures

Attack: Inspect function sources, property descriptors, or prototype chains to detect that APIs have been wrapped.

Examples include:

- checking `Function.prototype.toString` outputs,
- checking `Object.getOwnPropertyDescriptor`,
- checking `hasOwnProperty` anomalies,
- comparing descriptor flags (writable, configurable, enumerable).

PAL status: Extension-based defenses inherently face this risk because they modify or wrap objects in JavaScript. PAL partially mitigates this by preserving API contracts, return types, and expected behavior. However, pure JS wrapping can be detectable by a sufficiently motivated attacker.

Mitigation direction:

- Minimize detectable wrapper artifacts by preserving descriptors as closely as possible.
- Use stealthy patching approaches that avoid altering easily inspected properties.
- Treat detection resistance as a first-class test suite: run common wrapper-detection probes on PAL.

10.2.3 D3: Detection via Cross-Context Inconsistency

Attack: Compare the same surface across top, iframe, and worker contexts within the same page session. If values differ in unexpected ways, the attacker flags defense presence.

PAL status: Results show 0.00 cross-context consistency for Canvas and WebGL in privacy mode. This can be interpreted in two ways:

- Intentional context-specific drift (privacy-max).
- Unintentional mismatch due to incomplete propagation or differing seeds.

Either way, it creates a potential detectability signal if a tracker expects the same device to report identical outputs across contexts.

Mitigation direction: Add an optional coherent privacy profile that shares drift seeds across contexts for selected surfaces. This allows a user to trade maximum privacy for reduced detectability risk.

10.2.4 D4: Detection via Timing Artifacts

Attack: Measure the runtime of surface calls and detect unusual overhead, especially if wrappers add measurable latency.

PAL status: Timing overhead increases significantly in compat and privacy modes relative to vanilla, especially in iframe and worker contexts. This overhead can be used as a side channel.

Mitigation direction:

- Cache per-epoch parameters so hooks do minimal work.
- Avoid expensive operations in frequently called hot paths.
- Use lazy evaluation so that heavy computations occur only if the surface is actually used.

10.3 Bypass Analysis

A tracker can attempt to bypass PAL by shifting its fingerprint method, changing contexts, or using alternative surfaces.

10.3.1 B1: Context Shifting

Attack: If top context is protected, move probing to iframes or workers.

PAL status: PAL reduces linkability in both iframes (0.27) and workers (0.07). This indicates that PAL already resists simple context shifting.

Residual risk: Iframe linkability remains higher than worker linkability, consistent with partial iframe drift rates. Improving iframe propagation is a direct way to reduce this bypass avenue.

10.3.2 B2: Surface Substitution

Attack: If Canvas/WebGL drift, substitute other surfaces, such as:

- font measurement via layout side channels,
- WebRTC candidate signals,
- performance and timing features,
- media device enumeration,
- battery or sensor APIs where available,
- subtle rendering differences in CSS or SVG pipelines.

PAL status: PAL currently focuses on a subset of high-leverage surfaces. This is reasonable because most practical fingerprints rely heavily on graphics and audio. However, a determined attacker may shift to alternative surfaces.

Mitigation direction:

- Expand PAL's surface coverage in a measured way, starting with surfaces that add high entropy and low breakage risk.
- Add a surface prioritization framework: entropy contribution, stability, hook feasibility, breakage risk.

10.3.3 B3: Fuzzy Matching Rather Than Strict Equality

Attack: Instead of strict equality of K, use probabilistic classification. Even if some surfaces drift, the attacker may still link a user based on partially stable signals.

PAL status: PAL's evaluation focuses on strict-equality linkability. This is a strong baseline but does not fully model advanced trackers.

Mitigation direction:

- Extend evaluation to fuzzy linkability: measure similarity distances between fingerprints across epochs and compute classification accuracy of a simple linkage model.
- Increase drift coverage so that the fingerprint's feature vector distribution shifts meaningfully across epochs.

This direction strengthens the research contribution because it moves beyond strict equality while still being grounded in measurable metrics.

10.4 Residual Linkability: What Still Links the User

Even in privacy mode, residual linkability can persist in certain contexts because:

- some surfaces are intentionally stable (Navigator, Screen, Intl),
- some drift is partial (iframe),
- some drift is missing (worker audio),
- some sites may not invoke drifted surfaces in a consistent way across epochs.

The key point is that residual linkability in PAL is not mysterious. It is traceable to measured drift coverage gaps. This is a strong property: it means improvements can be targeted and validated.

10.5 Robustness Properties and Engineering Hardening

A privacy defense only matters if it remains deployable. The following robustness properties are important.

10.5.1 R-API: Preserve API Contracts

All wrapped functions must:

- accept the same argument types,
- return values of the same type and shape,
- preserve thrown error types where possible,
- preserve property descriptors as closely as feasible.

Contract violations are a major source of breakage and detection.

10.5.2 R-Bounds: Bound Perturbations

Perturbations must be bounded and plausible. The goal is to shift hashes, not to produce absurd or visibly wrong outputs.

10.5.3 R-Determinism: Enforce Intra-Epoch Stability

Within an epoch, the same call under the same persona should return identical outputs. This is both a usability property and a detection resistance property.

10.5.4 R-Fail-Safe: Safe Fallback on Exceptions

Hook boundaries should:

- catch exceptions,
- revert to original behavior if the hook fails,
- log failures for debugging.

A fail-safe mechanism prevents a single unhandled edge case from breaking entire pages.

10.5.5 R-Observability: Logging Integrity

Logging must:

- not leak the persona seed or sensitive internals,
- remain structured and parseable,
- capture enough fields to reproduce metric computations,
- distinguish site baseline errors from hook-related errors where possible.

10.6 Hardening Roadmap

Based on this security analysis, the most valuable hardening work items are:

- Add a wrapper-detection test suite to continuously evaluate detectability risks.
- Add a coherent privacy profile option for cross-context coherence on selected surfaces.
- Improve iframe propagation and worker audio coverage to reduce residual linkability.
- Optimize hot hook paths and cache drift parameters to reduce timing side channels.
- Expand surface coverage based on a principled entropy-versus-breakage prioritization.

10.7 Summary

This chapter analyzed PAL under realistic detection and bypass strategies. PAL resists naive instability detection by using deterministic drift rather than per-call randomness. Residual detectability risks remain due to wrapper artifacts, cross-context inconsistencies, and timing overhead. PAL already resists simple context shifting by sharply reducing composite linkability in iframe and worker contexts, but iframe residual linkability and worker audio drift remain the clearest bypass targets. The chapter identified concrete robustness properties and a hardening roadmap that directly maps to measurable gaps found in the evaluation.

CHAPTER 11

PROBLEM DEFINITION AND FORMAL METRICS

11.1 Why Fingerprinting Remains a Tracking Primitive

Web tracking is often discussed through the lens of storage, such as cookies, localStorage, or other persistent client-side identifiers. However, even when storage mechanisms are restricted or cleared, tracking can remain possible through fingerprinting. In fingerprinting, a script derives an identifier from observable properties of the browser environment. These properties may be stable for long periods, and they can be collected repeatedly across sessions.

For PAL, the core privacy problem is not whether a single API can be perturbed once, but whether the **overall fingerprint vector remains reusable across time**. Therefore, this report formalizes the privacy objective in terms of **cross-epoch linkability** and measures it through drift and composite matching metrics.

11.2 Core Entities and Notation

This section defines the entities used throughout the report.

11.2.1 Modes

Each run is conducted under a mode $m \in \mathcal{M}$:

$$\mathcal{M} = \{\text{vanilla},\ \text{compat},\ \text{privacy}\}$$

11.2.2 Sites

Let $\mathcal{S}$ denote the set of evaluated sites. Each site is indexed by $s \in \mathcal{S}$.

11.2.3 Epochs

An epoch is a discrete logical time period during which PAL aims to keep outputs stable in privacy mode. Let:

$$e \in \mathcal{E} = \{1, 2, \ldots, E\}$$

In the evaluation conducted in this report, $E = 3$.

11.2.4 Execution Contexts

PAL treats execution context as a first-class dimension because fingerprinting can occur outside the top frame. Let:

$$c \in \mathcal{C} = \{\text{top},\ \text{iframe},\ \text{worker}\}$$

11.2.5 Fingerprint Surfaces

A fingerprint surface is an API family or measurement channel that produces observable output used in fingerprinting. Let:

$$x \in \mathcal{X}$$

In this report, the primary surfaces considered include:

$$\mathcal{X} = \{\text{Canvas},\ \text{WebGL},\ \text{Audio},\ \text{Navigator},\ \text{Screen},\ \text{Intl}\}$$

Not all surfaces need to be perturbed in privacy mode. PAL focuses on high-entropy surfaces to reduce composite linkability while maintaining plausibility.

11.2.6 Fingerprint Output

For a given site s, epoch e, context c, mode m, and surface x, the measurement produces an output:

$$V_{s,e,c,m}(x)$$

This output may be a hash of raw data, a normalized vector, or a structured value derived from the corresponding surface.

11.3 Composite Fingerprint Key

Trackers rarely rely on only one surface. Instead, they combine multiple surfaces into a composite identifier. PAL therefore evaluates the composite key defined as:

$$K_{s,e,c,m} = \big(V_{s,e,c,m}(\text{Canvas}),\ V_{s,e,c,m}(\text{WebGL}),\ V_{s,e,c,m}(\text{Audio}),\ V_{s,e,c,m}(\text{Navigator}),\ V_{s,e,c,m}(\text{Screen})\big)$$

This composite key K captures the strict-equality scenario: if all components match across epochs, a tracker can reuse K as a stable identifier.

11.4 Problem Statement

11.4.1 Tracking via Stable Fingerprints

Given a user and a browser environment, an attacker aims to compute a stable identifier I such that:

$$I_{t_1} \approx I_{t_2}$$

for the same user across time, while remaining sufficiently unique across users.

In a simplified model, assume the attacker uses the composite fingerprint key K as the identifier:

$$I \equiv K$$

The tracking problem exists when K remains unchanged across time, meaning:

$$K_{s,e_1,c,m} = K_{s,e_2,c,m} \quad \text{for } e_1 \neq e_2$$

11.4.2 PAL Objective

PAL aims to reduce the reuse of K across epochs in privacy mode while preserving stability within an epoch. The primary objective is:

- **Intra-epoch stability:** for a fixed epoch e, repeated measurements should not introduce unnecessary randomness.
- **Inter-epoch drift:** for privacy mode, selected high-entropy surfaces should change across epochs to reduce composite matching.

In formal terms, PAL seeks to reduce:

$$\Pr\left(K_{s,e_1,c,\text{privacy}} = K_{s,e_2,c,\text{privacy}}\right) \quad \text{for } e_1 \neq e_2$$

11.5 Formal Metrics

This section defines the exact metrics reported in the evaluation.

11.5.1 Inter-Epoch Drift Rate

For a fixed surface x, drift rate measures how often the surface output changes between epochs.

For a given site s, context c, and mode m, define the pairwise drift indicator:

$$D_{s,(e_i,e_j),c,m}(x) = \begin{cases} 1 & \text{if } V_{s,e_i,c,m}(x) \neq V_{s,e_j,c,m}(x) \\ 0 & \text{otherwise} \end{cases}$$

Let $\mathcal{P}$ be the set of unordered epoch pairs:

$$\mathcal{P} = \{(e_i, e_j) : 1 \leq i < j \leq E\}$$

Then the drift rate is:

$$\text{DriftRate}_{c,m}(x) = \frac{\sum_{s\in\mathcal{S}} \sum_{(e_i,e_j)\in\mathcal{P}} D_{s,(e_i,e_j),c,m}(x)}{\sum_{s\in\mathcal{S}} |\mathcal{P}|}$$

Interpretation:

- DriftRate $= 0.00$ means the surface is fully stable across epochs.
- DriftRate ≈ 1.00 means the surface changes across epochs in nearly all comparisons.

11.5.2 Cross-Context Consistency Rate

Cross-context consistency captures whether a surface output matches across contexts within the same epoch. This metric is relevant because a tracker may compare signals gathered in different contexts (for example, top frame versus iframe) to detect spoofing or to increase confidence.

For a fixed surface x and mode m, define the within-epoch consistency indicator for a site s and epoch e:

$$C_{s,e,m}(x) = \begin{cases} 1 & \text{if } V_{s,e,\text{top},m}(x) = V_{s,e,\text{iframe},m}(x) = V_{s,e,\text{worker},m}(x) \\ 0 & \text{otherwise} \end{cases}$$

Then:

$$\text{ConsistencyRate}_m(x) = \frac{\sum_{s\in\mathcal{S}} \sum_{e\in\mathcal{E}} C_{s,e,m}(x)}{|\mathcal{S}| \cdot |\mathcal{E}|}$$

Interpretation:

- ConsistencyRate $= 1.00$ means the surface matches across contexts within epochs.
- ConsistencyRate $= 0.00$ means the surface differs across contexts.

11.5.3 Composite Linkability

Composite linkability directly models the attacker's success condition for strict equality reuse of a fingerprint key.

For a site s, context c, and mode m, define the composite match indicator:

$$L_{s,(e_i,e_j),c,m} = \begin{cases} 1 & \text{if } K_{s,e_i,c,m} = K_{s,e_j,c,m} \\ 0 & \text{otherwise} \end{cases}$$

Then the composite linkability metric is:

$$\text{Linkability}_{c,m} = \frac{\sum_{s\in\mathcal{S}} \sum_{(e_i,e_j)\in\mathcal{P}} L_{s,(e_i,e_j),c,m}}{\sum_{s\in\mathcal{S}} |\mathcal{P}|}$$

Interpretation:

- Linkability $= 1.00$ means fingerprints are fully reusable across epochs.
- Linkability $= 0.00$ means fingerprints are never identical across epochs.

11.5.4 Error Signal Metrics

Error events can indicate interference between instrumentation and real site scripts. However, because modern sites often emit baseline errors even without PAL, errors are treated as a coarse stability signal rather than a definitive breakage metric.

For each mode m, define:

- **Total error events:** the count of captured error events aggregated across the crawl.
- **Sites with any error:** the number of sites where at least one error event occurred.

These are reported as descriptive statistics to guide robustness work.

11.5.5 Timing Overhead Metrics

Timing captures the runtime cost of PAL instrumentation and drift logic.

For each context c and mode m, the crawl records probe time samples $T_{i,c,m}$. The report summarizes these with mean and standard deviation:

$$\mu_{c,m} = \text{mean}(T_{i,c,m}), \quad \sigma_{c,m} = \text{std}(T_{i,c,m})$$

Timing results support optimization planning and help quantify deployability constraints.

11.6 Success Criteria by Mode

PAL has different success criteria depending on mode.

11.6.1 Vanilla

Vanilla is not intended to provide privacy. It is the baseline. Expected:

- DriftRate near 0.00 for most surfaces.
- Linkability near 1.00 across contexts.

11.6.2 Compatibility Mode

Compatibility mode targets stability and coherence:

- DriftRate should remain near 0.00.
- Linkability is expected to remain high.
- Cross-context consistency should remain high for persona-linked surfaces.

11.6.3 Privacy Mode

Privacy mode targets tracking disruption:

- DriftRate should be high for high-entropy surfaces (Canvas, WebGL, and where applicable Audio).
- Linkability should be low, ideally approaching 0.00 in all contexts.
- Intra-epoch stability should still hold, so drift is epoch-based rather than per-call random.

11.7 Summary

This chapter formalized the fingerprint tracking problem and defined the metrics used to evaluate PAL. It introduced notation for sites, epochs, contexts, and surfaces, defined a composite fingerprint key to model strict-equality fingerprint reuse, and provided formal definitions for inter-epoch drift rate, cross-context

consistency, composite linkability, and deployability signals such as errors and overhead. These definitions establish the foundation for the later design and evaluation chapters, where PAL's mechanisms are analyzed in terms of measurable changes to drift and linkability under the attacker model defined previously.

CHAPTER 12

DESIGN PRINCIPLES AND SYSTEM INVARIANTS

12.1 Why Design Principles Matter for a Fingerprinting Defense

A fingerprinting defense is not only judged by whether it changes a value. It is judged by whether it produces a browser environment that remains coherent, plausible, and stable enough to avoid obvious detection, while still reducing the attacker's ability to reuse identifiers across time.

PAL is therefore built around explicit design principles and invariants. These invariants serve two purposes:

- They define what PAL guarantees in each mode, which makes the system testable and the claims defensible.
- They expose the core tradeoffs, so that limitations are framed as engineering targets rather than conceptual failure.

This chapter documents the design principles behind PAL and formalizes the invariants that PAL attempts to satisfy.

12.2 High-Level Design Principles

12.2.1 DP1: Composite Tracking Requires Composite Defenses

Modern fingerprinting does not rely on a single surface. A tracker typically collects multiple surfaces and combines them into a composite identifier. Therefore, PAL focuses on reducing **composite equality across epochs** instead of over-optimizing a single surface in isolation.

This principle directly motivates the composite linkability metric defined earlier.

12.2.2 DP2: Drift Must Be Controlled, Not Random

Naive privacy tools often inject per-call randomness. This makes a browser unstable inside a single session, increases breakage risk, and creates a detectable signature because outputs fluctuate in ways that real devices typically do not.

PAL instead uses **controlled drift**:

- stable within an epoch (so a site sees consistent behavior),
- variable across epochs (so the fingerprint is less reusable over time).

This principle motivates deterministic epoch-seeded perturbations rather than unbounded randomness.

12.2.3 DP3: Persona Coherence is a First-Class Constraint

A browser environment contains many correlated values. If a privacy tool modifies a subset of them incorrectly, it can create internal contradictions that are easier to detect than the original fingerprint.

PAL therefore treats persona coherence as a first-class constraint. Even when privacy mode introduces drift, the drift must remain bounded and must not break coarse relationships between surfaces.

12.2.4 DP4: Context Coverage is Required

Fingerprinting can occur in the top frame, in third-party iframes, and in background workers. A defense that operates only in the top frame can leave measurable holes that allow a tracker to recover stable identifiers from other contexts.

PAL therefore aims to provide drift and persona enforcement across:

$$\{\text{top},\ \text{iframe},\ \text{worker}\}$$

and evaluates all three explicitly.

12.2.5 DP5: Deployability Under Real Browser Constraints

PAL is implemented under real deployment constraints (Chrome extension model, MV3 execution model, and the separation between extension worlds and page worlds). A design that assumes privileged browser modifications is not deployable for real users.

PAL's mechanisms are therefore chosen to be implementable in the real extension model, using injection, message passing, and interception strategies that work at the JavaScript level.

12.3 System Invariants

This section defines the invariants PAL aims to satisfy. Some invariants apply to all modes, while others are mode-specific.

12.3.1 Invariant I1: Intra-Epoch Stability

Within a fixed epoch e, a fingerprint surface output should remain stable under repeated queries, up to bounded and deterministic transformations. Informally, if a site calls the same surface multiple times in the same epoch, the response should not exhibit noisy fluctuation that resembles random jitter.

Formally, for a given site s, context c, mode m, and surface x, PAL aims to satisfy:

$$V_{s,e,c,m}(x) \approx V'_{s,e,c,m}(x)$$

for repeated measurements inside epoch e.

In vanilla and compatibility modes, this invariant reduces to direct stability (no PAL-induced drift). In privacy mode, stability is achieved by deterministic epoch-seeded behavior rather than per-call randomization.

12.3.2 Invariant I2: Inter-Epoch Drift for Selected Surfaces in Privacy Mode

In privacy mode, PAL aims to introduce inter-epoch change on selected high-entropy surfaces. For those surfaces, PAL aims for:

$$V_{s,e_1,c,\text{privacy}}(x) \neq V_{s,e_2,c,\text{privacy}}(x) \quad \text{for many pairs } e_1 \neq e_2$$

The exact target drift rate depends on surface and context. The goal is not necessarily to force 1.00 drift everywhere, but to reduce the attacker's ability to reuse the composite fingerprint key.

12.3.3 Invariant I3: Mode Separation

Each mode must have a clearly different behavior profile:

- **Vanilla:** no intervention; surfaces behave as in a standard browser.
- **Compatibility:** persona coherence enforced without time drift.
- **Privacy:** epoch-seeded drift introduced on selected surfaces while preserving intra-epoch stability.

This invariant ensures the user meaningfully controls tradeoffs.

12.3.4 Invariant I4: Persona Coherence Across Surfaces

PAL aims to avoid contradictory combinations of values that would not appear in a real browser environment. Persona coherence is enforced at two levels:

- **Static coherence:** stable persona fields (languages, platform profile, high-level identity fields) remain consistent.
- **Dynamic coherence:** drifted surfaces remain within plausible ranges and do not violate coarse constraints.

For example, if a platform persona implies certain ranges or behaviors, drifted values should not create extreme outliers that trivially reveal spoofing.

12.3.5 Invariant I5: Context Robustness

For a given mode m and epoch e, PAL aims to minimize unintended divergence across contexts. This does not mean that all contexts must match perfectly for all surfaces, but it means that:

- A surface that is intended to be stable should not silently diverge across contexts.
- A surface that is intended to drift should drift consistently across contexts, subject to context-specific implementation constraints.

In other words, PAL aims to avoid having "privacy mode" behave like privacy in one context and vanilla in another.

12.3.6 Invariant I6: Bounded Perturbations

In privacy mode, drift is introduced through perturbation. Perturbations must be bounded:

- They must not produce invalid outputs that break the API contract.
- They must not exceed plausible ranges that would instantly flag spoofing.
- They must be deterministic for a given epoch and persona, ensuring repeatability.

Bounded perturbation is a core technique for preserving plausibility while still reducing fingerprint reuse.

12.4 Non-Goals as Design Constraints

Explicit non-goals are also part of the design, because they prevent the system from drifting into unbounded complexity.

12.4.1 NG1: Perfect Anonymity

PAL does not claim to provide perfect anonymity. It reduces fingerprint-based linkability across epochs, but it does not eliminate all tracking vectors, especially server-side identifiers, network-level correlation, or behavioral profiling.

12.4.2 NG2: Defeating Dedicated Anti-Fraud Fingerprinting at All Costs

PAL prioritizes privacy while preserving usability. Some high-security anti-fraud systems may attempt deeper probing or enforce strict consistency checks. PAL does not attempt to win all adversarial arms races in the first iteration, but it is designed to be extensible.

12.4.3 NG3: Breaking Web Compatibility for Maximum Drift

A design that maximizes drift without constraints can destroy usability. PAL instead aims to operate under a realistic user constraint: the browser must remain usable for real sites, and privacy must be achieved with controlled drift on selected surfaces.

12.5 Design Tradeoffs and Rationale

12.5.1 Tradeoff T1: Drift Strength Versus Plausibility

Higher drift strength reduces linkability but may increase detectability and breakage. PAL therefore applies strong drift primarily to high-entropy surfaces where small perturbations yield high identifier change.

12.5.2 Tradeoff T2: Context Uniformity Versus Deployment Complexity

Ensuring uniform drift across top, iframe, and worker contexts requires careful injection and propagation. PAL prioritizes deployability first, then iteratively improves uniformity as gaps are identified.

12.5.3 Tradeoff T3: Performance Versus Coverage

Hooking more surfaces and performing more computations increases overhead. PAL's architecture therefore separates core high-impact surfaces from optional surfaces, and future optimization aims to reduce overhead without reducing privacy impact.

12.6 How These Invariants Will Be Validated

The metrics defined earlier map directly to these invariants:

- **Intra-epoch stability (I1)** is checked through repeated probe behavior within the same epoch and through low drift where expected.
- **Inter-epoch drift (I2)** is measured by drift rates on selected surfaces.
- **Mode separation (I3)** is confirmed by contrasting drift and linkability across vanilla, compatibility, and privacy.
- **Context robustness (I5)** is measured by cross-context consistency and by comparing drift/linkability across contexts.
- **Deployability constraints** are reflected in timing overhead and error signals.

12.7 Summary

This chapter established PAL's design principles and formal system invariants. PAL is built around controlled epoch-based drift rather than per-call randomness, prioritizes composite fingerprint disruption, enforces persona coherence, and treats context coverage as essential. These invariants define what PAL guarantees in each mode and motivate the evaluation metrics used in later chapters. The next chapter will describe PAL's architecture and implementation under real browser constraints, showing how these invariants are realized in a deployable system.

CHAPTER 13

DETERMINISTIC DRIFT MECHANISM

13.1 Why Deterministic Drift is the Core Contribution

Most simplistic anti-fingerprinting approaches attempt to add random noise. While this may reduce repeatability in a narrow sense, it typically introduces two major failures:

- **Intra-session instability:** values fluctuate across repeated calls inside the same session, which breaks scripts that assume determinism and also increases detectability because real devices do not behave like continuous randomness generators.
- **Lack of reproducibility:** random noise makes evaluation and debugging difficult because identical runs do not produce identical traces.

PAL uses a different approach: **deterministic drift**. Deterministic drift is designed to satisfy two requirements simultaneously:

- **Stability within an epoch:** repeated queries inside the epoch yield stable outputs.
- **Change across epochs:** once the epoch changes, selected fingerprint surfaces shift, reducing the attacker's ability to reuse a composite fingerprint across time.

This chapter formalizes how deterministic drift is constructed, how it is applied to different surfaces, and what constraints are enforced to keep perturbations plausible and bounded.

13.2 Design Requirements

PAL's drift mechanism is designed to satisfy the following requirements.

13.2.1 R1: Intra-Epoch Determinism

For a given persona, epoch, and context, all perturbations must be deterministic. If a script calls a surface multiple times in the same epoch, PAL should not introduce per-call jitter.

13.2.2 R2: Inter-Epoch Separation

For privacy mode, the drift mechanism must ensure that the derived perturbation parameters change when the epoch changes. Otherwise, fingerprints remain reusable across epochs.

13.2.3 R3: Surface Separation

Perturbations for different surfaces must be decoupled. If Canvas and WebGL drift always move together in a perfectly correlated way, a tracker may exploit correlation patterns to detect spoofing or to reduce uncertainty.

13.2.4 R4: Context Awareness

Because PAL evaluates and defends across top/iframe/worker contexts, the drift mechanism must support context-specific derivations without silently collapsing into a single global behavior that fails in one context.

13.2.5 R5: Boundedness and Plausibility

Perturbations must preserve API contracts and remain within plausible ranges. Drift must not create absurd outputs that trivially reveal modification or break functionality.

13.3 Seed Hierarchy

PAL uses a hierarchical seed derivation model. This section defines the conceptual structure.

13.3.1 Base Persona Seed

Let P be the base persona seed. This is an internal value that remains stable unless explicitly reset by the user or by a configuration change.

The persona seed provides two key properties:

- **Consistency:** within a given persona profile, outputs remain coherent.
- **Separation across users/personas:** different personas yield different derived perturbations.

13.3.2 Epoch Identifier

Let e be the epoch identifier. In PAL, the epoch can be defined in multiple ways depending on the evaluation configuration:

- **Time-based epoch:** epoch changes after a fixed time interval.
- **Run-based epoch:** epoch changes per evaluation run (useful in crawling experiments).
- **Event-based epoch:** epoch changes under explicit triggers (future work).

In the evaluation presented in this report, $E = 3$ epochs were used to measure inter-epoch drift behavior.

13.3.3 Surface Label and Context Key

Let x be the surface label, such as `Canvas` or `WebGL`. Let c be the context key in $\{\text{top}, \text{iframe}, \text{worker}\}$.

These values ensure separation:

- Different surfaces derive different seeds.
- Different contexts can derive different seeds if required.

13.3.4 Derived Surface Seed

For each surface and context, PAL conceptually derives a surface seed:

$$S_{x,c,e} = H\left(P \| e \| x \| c \| \text{opt}\right)$$

Where:

- $H(\cdot)$ is a cryptographic hash or strong mixing function.
- $\|$ denotes concatenation.
- `opt` denotes optional stabilizers or subkeys (discussed below).

The most important property is that:

- Holding P, x, c fixed while changing e changes $S_{x,c,e}$.
- Holding P, e, c fixed while changing x changes $S_{x,c,e}$.

This ensures inter-epoch separation and surface separation.

13.4 Optional Stabilizers and Subkeys

In some cases, PAL can optionally include additional keys in seed derivation. These are used to control correlation and avoid pathological behavior.

Examples of optional stabilizers include:

- **Frame identity:** a stable identifier for the frame instance to avoid accidental collisions.
- **Origin or site key:** in some designs, drift can be site-scoped to prevent cross-site correlation patterns. This is treated as an optional design axis rather than a default.
- **Callsite class:** a class label for different sub-surfaces inside a surface family (for example, separating WebGL parameters from WebGL render outputs).

In this report's evaluation design, PAL focuses primarily on epoch-based drift rather than per-site isolation. However, the architecture supports extending seed derivation with additional keys.

13.5 Deterministic Pseudorandom Number Generation

Once a surface seed $S_{x,c,e}$ is derived, PAL needs to produce structured perturbation parameters. This requires a deterministic pseudorandom number generator (PRNG) keyed by $S_{x,c,e}$.

Let $\mathsf{PRNG}(S)$ produce a stream of pseudorandom values:

$$r_1, r_2, r_3, \ldots$$

Because the PRNG is deterministic, the same $S_{x,c,e}$ yields the same stream inside the epoch. PAL uses this stream to derive:

- small pixel offsets or noise masks (Canvas),
- small parameter shifts or string perturbations (WebGL),
- small deterministic transformations of audio output or derived hashes (Audio),
- bounded perturbations for any additional targeted surfaces.

13.6 Perturbation Construction Strategy

This section describes the general approach used by PAL for constructing perturbations.

13.6.1 Step 1: Derive Seed

Compute $S_{x,c,e}$ from (P, e, x, c).

13.6.2 Step 2: Initialize PRNG

Initialize a PRNG stream keyed by $S_{x,c,e}$.

13.6.3 Step 3: Generate Perturbation Parameters

Consume values from the PRNG stream and map them into bounded perturbation parameters θ.

$$\theta_{x,c,e} = f\left(\mathsf{PRNG}(S_{x,c,e})\right)$$

Where $f(\cdot)$ enforces boundedness, plausibility, and correct parameter types.

13.6.4 Step 4: Apply Perturbation at Hook Sites

When a script queries the surface x, PAL intercepts the call and applies the transformation:

$$\widetilde{V}_{s,e,c,\text{privacy}}(x) = g\left(V_{s,e,c,\text{vanilla}}(x), \theta_{x,c,e}\right)$$

Where:

- V is the baseline surface output.
- $\widetilde{V}$ is the modified output.
- $g(\cdot)$ is a surface-specific perturbation function.

In compatibility mode, the same framework exists but drift parameters are disabled or forced to identity values, yielding stability.

13.7 Surface-Specific Strategies

This section explains the main targeted surfaces.

13.7.1 Canvas Drift Strategy

Canvas fingerprinting commonly relies on:

- rendering text, shapes, or gradients,
- reading pixel data via `getImageData` or `toDataURL`,
- hashing the returned pixel buffer.

PAL's objective is to cause the derived canvas fingerprint hash to change across epochs while keeping rendering plausible.

A common and practical approach is to apply a small deterministic perturbation to pixel buffers at read-out time. Conceptually:

- Intercept `getImageData` and `toDataURL`.
- Construct a deterministic noise mask based on $\theta_{\text{Canvas},c,e}$.
- Apply the mask to a small subset of pixels or apply tiny channel shifts within bounds.

A key design constraint is that perturbation should not visually degrade normal content significantly. Therefore, PAL's perturbation is bounded and applied in a way that affects hash stability more than human perception.

13.7.2 WebGL Drift Strategy

WebGL fingerprinting relies on a combination of:

- renderer and vendor strings,
- supported extensions,
- floating point precision and shader behavior,
- deterministic rendering outputs of scenes.

PAL targets WebGL by intercepting relevant access points and introducing bounded perturbations. The drift must remain coherent so that WebGL continues to function.

Possible perturbation forms include:

- deterministic perturbation of selected queried parameters (bounded changes),

- controlled modifications of returned strings consistent with persona,
- perturbation of readback buffers in a similar style to Canvas for render output hashes.

The exact hook points depend on implementation details, but the conceptual goal is the same: change the derived fingerprint across epochs while maintaining internal consistency.

13.7.3 Audio Drift Strategy

Audio fingerprinting often uses:

- offline audio contexts to generate deterministic signals,
- FFT or time-domain analysis,
- hashing of resulting arrays.

PAL introduces deterministic drift in privacy mode by applying bounded transformations that shift the derived hash across epochs while preserving stability within an epoch.

Unlike Canvas and WebGL, Audio drift is more sensitive to implementation differences across contexts. The evaluation results in later chapters show strong drift in top and iframe contexts but a current gap in worker context. This supports the importance of context-aware hooking and validates the evaluation's multi-context design.

13.8 Preventing Common Failure Modes

This section documents failure modes that deterministic drift must avoid.

13.8.1 FM1: Per-Call Randomness Signatures

If outputs vary on every call, a tracker can detect it by repeated measurement. PAL avoids this by deriving all perturbations deterministically from the epoch seed.

13.8.2 FM2: Unbounded or Invalid Outputs

If perturbations produce invalid values, sites break and spoofing becomes obvious. PAL enforces boundedness and preserves output types.

13.8.3 FM3: Cross-Surface Correlation Artifacts

If different surfaces drift in perfectly correlated patterns, detection becomes easier. PAL separates seeds by surface labels and can optionally introduce independent subkeys.

13.8.4 FM4: Context Collapse

If privacy mode works only in one context, attackers can shift measurement to another context. PAL therefore treats context as part of seed derivation and attempts to propagate policy across contexts.

13.9 Summary

This chapter formalized PAL's deterministic drift mechanism. PAL derives surface-specific, context-aware seeds from a stable persona seed and an epoch identifier, initializes a deterministic PRNG, and generates bounded perturbation parameters that remain stable within an epoch but shift across epochs. Drift is applied through surface interceptors for high-entropy surfaces such as Canvas and WebGL, with additional support for Audio. The mechanism is designed to prevent common failure modes of naive noise injection while remaining deployable under real browser extension constraints. The next chapter will describe the implementation in concrete terms, including hook points, injection strategy, context propagation mechanisms, and telemetry integration.

CHAPTER 14

DISCUSSION, LIMITATIONS, AND FUTURE WORK

14.1 Discussion Framed by the Threat Model

PAL is evaluated against a strict-equality fingerprint reuse attacker model: the attacker collects a composite fingerprint key K from multiple high-entropy surfaces and attempts to reuse it across time. Under that model, PAL's most important empirical outcome is the reduction of composite linkability in privacy mode across contexts.

The results show three high-level points:

- **The deterministic drift concept works at scale on real sites.** Canvas and WebGL drift strongly across epochs in privacy mode and remain stable in vanilla and compatibility modes.
- **Composite linkability is sharply reduced.** Privacy mode reduces strict-equality linkability to 0.00 in the top context and substantially reduces it in iframe and worker contexts.
- **Context-level coverage gaps are visible and measurable.** Audio drift is not yet achieved in worker context, and iframe drift is partial across multiple surfaces.

These observations support the central claim: PAL is not merely adding noise, it enforces epoch-based deterministic drift on high-impact surfaces in a way that measurably reduces fingerprint reuse across time.

14.2 Interpreting the Core Results

14.2.1 Why Top-Frame Linkability Reaches Zero

Top context is where PAL has the cleanest control plane and earliest injection. As a result:

- hook installation occurs reliably,
- epoch and persona inputs are consistent,
- targeted surfaces drift strongly across epochs.

Under these conditions, the composite key K rarely survives unchanged across epochs, so linkability falls to 0.00 in top context.

This result is important because top-frame fingerprinting remains a common and practical tracking strategy. Achieving zero strict-equality linkability in the top frame is therefore a high-confidence signal of privacy impact.

14.2.2 Why Iframe Linkability Remains Non-Zero

Iframe context shows reduced but non-zero linkability (0.27). This aligns with partial iframe drift for Canvas, WebGL, and Audio.

There are multiple plausible contributors:

- **Propagation timing:** iframe creation is dynamic, and hooks may not always install early enough for all frames.
- **Origin constraints:** third-party frames may have differing injection constraints depending on browser security model.
- **Probe variability:** some sites may construct frames differently across epochs, affecting which hook paths are executed.

Importantly, iframe linkability is still far below the baseline (1.00), indicating that the system already disrupts many iframe fingerprints. The residual linkability provides a concrete engineering objective: improve iframe hook completeness and ensure drift activation is uniform.

14.2.3 Why Worker Linkability is Very Low Despite Worker Audio Gap

Worker context linkability falls to 0.07, which is strong, especially given the observed lack of audio drift in worker context.

This suggests that:

- Canvas and WebGL drift in worker context is sufficiently strong to disrupt most composite keys.
- Even without audio drift, the composite equality condition often fails because other key surfaces are not stable across epochs.

This strengthens the claim that PAL's privacy benefit is not dependent on a single surface. The system derives value from multi-surface drift, and a strong subset of surfaces can still reduce composite linkability significantly.

14.3 Limitations

This section documents limitations in a way that preserves credibility without weakening the core claim.

14.3.1 L1: Worker Audio Drift is Not Yet Implemented End-to-End

Audio drift is strong in top context, partial in iframe, and absent in worker. This indicates that worker audio instrumentation requires additional coverage work.

This limitation does not invalidate PAL's core privacy outcome because worker composite linkability is already low. However, it represents an important completeness milestone, since attackers can shift probing into workers.

14.3.2 L2: Cross-Context Coherence is Not Enforced for Some Drifted Surfaces

Cross-context consistency for Canvas and WebGL in privacy mode is 0.00. This can occur either because PAL deliberately allows context-specific drift or because drift differs unintentionally across contexts.

This is not automatically a flaw. In some designs, context-specific variation can increase privacy by preventing cross-context correlation. However, it can also increase detectability if a site expects identical behavior across contexts.

Therefore, this limitation is best treated as a tunable design axis:

- privacy strength may increase with context-specific drift,
- detectability risk may decrease with cross-context coherence.

Future versions of PAL can optionally support a "coherent privacy" profile where drift parameters are shared across contexts for specific surfaces.

14.3.3 L3: Performance Overhead is Non-Trivial and Highly Variable

Compatibility and privacy modes introduce overhead relative to vanilla. The overhead variance is high in iframe and worker contexts.

This limitation indicates that:

- some sites trigger expensive probe or hook paths,
- some contexts incur greater message-passing or instrumentation overhead.

Performance does not invalidate the privacy effect, but it affects deployability and usability. Therefore, optimization is a key next step.

14.3.4 L4: Error Events Increase Under Hooking

Captured error events increase in compatibility and privacy relative to vanilla. However, vanilla already exhibits baseline errors across many sites, so error counts must be interpreted as coarse signals.

This limitation suggests that PAL's interception layer can still trigger edge-case interactions on some sites, motivating improved defensive coding and narrower hook scopes for certain APIs.

14.3.5 L5: Dataset Size and Coverage

This evaluation used 19 sites and three epochs. While sufficient to show clear mode separation and strong linkability reduction, a larger crawl would provide:

- broader site diversity,
- stronger coverage of edge-case scripts,
- better statistical confidence bounds.

PAL's logging and evaluation pipeline is already built to support scaling to larger crawls without changing the conceptual framework.

14.4 Future Work Roadmap

This section outlines concrete, technically grounded improvements that directly follow from the measured gaps.

14.4.1 FW1: Worker Audio Instrumentation

Goal: ensure privacy-mode drift for audio fingerprinting in worker context.

Approach:

- Identify worker-visible audio pathways and ensure hooks are installed in worker execution environments.
- Ensure the worker receives consistent epoch and persona seeds.
- Apply deterministic drift at the same point in the audio pipeline as in top context, preserving boundedness.
- Re-run evaluation and confirm worker audio drift becomes non-zero, ideally approaching top-context behavior.

Expected impact:

- further reduction of worker composite linkability,
- improved completeness against attacker strategies that shift probing to workers.

14.4.2 FW2: Improve Iframe Drift Uniformity

Goal: reduce iframe residual linkability by improving hook completeness and drift activation.

Approach:

- Ensure hooks inject reliably into dynamically created frames.
- Verify that epoch and persona policy propagates without race conditions.
- Add targeted telemetry markers for injection success and drift activation.

Expected impact:

- higher drift rates in iframe contexts,
- reduced iframe composite linkability toward the top-context target.

14.4.3 FW3: Optional Cross-Context Coherent Privacy Profile

Goal: reduce detectability risk for environments that expect consistent cross-context behavior.

Approach:

- Allow configuration to share surface seeds across contexts for selected surfaces.
- Maintain deterministic drift across epochs, but unify within-epoch values across contexts.

This would allow users to choose between:

- **privacy-max profile:** context-specific drift, potentially stronger privacy.
- **coherent profile:** cross-context coherence, potentially lower detectability risk.

14.4.4 FW4: Performance Optimization

Goal: reduce mean overhead and reduce tail latency variability.
Approach:

- Cache derived perturbation parameters $\theta_{x,c,e}$ so they are computed once per epoch and reused.
- Reduce heavy operations inside hot hook paths, especially on frequently-called functions.
- Minimize message passing across extension boundaries for high-frequency calls.

Expected impact:

- lower mean timing overhead,
- reduced standard deviation, particularly in iframe and worker contexts.

14.4.5 FW5: Expand Evaluation Scale and Add Confidence Reporting

Goal: strengthen evidence and improve scientific rigor.
Approach:

- Expand crawl from 19 sites to a larger sample.
- Increase epoch count to observe longer drift patterns.
- Add confidence intervals or bootstrap estimates for drift and linkability rates.

This does not change the core framework, it strengthens the statistical credibility of the results.

14.5 Broader Implications

PAL demonstrates that it is possible to design a deployable anti-fingerprinting tool that avoids per-call randomness and instead applies deterministic drift across controlled epochs. This approach has two meaningful implications:

- It provides a more realistic balance between privacy and usability than naive noise injection.
- It yields a research framework where privacy claims are directly tied to measurable cross-epoch linkability outcomes.

The results also show the value of explicitly measuring context behavior. Many defenses implicitly assume top-frame execution, but PAL's evaluation confirms that real privacy requires iframe and worker coverage to be treated as first-class concerns.

14.6 Summary

This chapter discussed PAL's results in terms of the threat model and system invariants. PAL achieves its strongest outcome in the top frame, eliminating strict-equality composite linkability across epochs in privacy mode. It also sharply reduces linkability in iframe and worker contexts, though iframe residual linkability and worker audio drift remain concrete coverage gaps. Performance overhead and error signals provide deployability targets rather than conceptual failures. The chapter outlined a future work roadmap focused on worker audio instrumentation, improved iframe propagation, optional cross-context coherent privacy, and performance optimizations. The next chapter concludes the report by summarizing the system's contributions and empirical evidence, and by stating clear claims supported by the evaluation.

CHAPTER 15

CONCLUSION

15.1 Summary of the Problem

Browser fingerprinting remains a practical and widely deployed tracking technique because it leverages stable, high-entropy signals exposed through JavaScript-accessible APIs. Unlike cookie-based tracking, fingerprinting can persist across sessions and can be collected from multiple execution contexts, including top frames, iframes, and workers. A realistic defense must therefore reduce an attacker's ability to reuse a composite fingerprint across time, while still keeping the browser environment coherent and usable on real sites.

15.2 What PAL Set Out To Do

PAL was built as a deployable, extension-based defense that targets the most leverage-heavy fingerprint surfaces using a controlled strategy:

- Maintain **intra-epoch stability** so that the browser remains consistent within a time window and does not exhibit per-call randomness.
- Introduce **inter-epoch deterministic drift** in privacy mode so that selected high-entropy surfaces change in a repeatable, bounded way as the epoch changes.
- Preserve a coherent persona and mode separation (vanilla, compatibility, privacy).
- Treat **context coverage** as first-class by evaluating top, iframe, and worker contexts.

15.3 Core Contributions

15.3.1 Contribution 1: Deterministic Epoch-Based Drift as a Deployable Defense Primitive

The first contribution is the use of deterministic drift across epochs rather than per-call randomness. PAL derives perturbations deterministically from a persona seed, an epoch identifier, a surface label, and a context key. This design preserves stability within an epoch while still breaking strict equality reuse across epochs in privacy mode.

15.3.2 Contribution 2: Multi-Context Evaluation as a Required Standard

The second contribution is treating iframe and worker execution contexts as a required evaluation dimension rather than an optional one. PAL's results demonstrate that privacy claims can look strong in top context while gaps can still exist in other contexts. Measuring all contexts makes those gaps visible and actionable.

15.3.3 Contribution 3: A Metric Framework Centered on Composite Linkability

The third contribution is the evaluation framing around composite linkability. Instead of reporting only per-surface changes, PAL evaluates whether a strict-equality composite fingerprint key remains reusable across epochs, which aligns with how practical tracking systems operate.

15.4 Empirical Results and Supported Claims

PAL was evaluated on a crawl of 19 sites across three modes (vanilla, compatibility, privacy), three epochs, and three contexts (top, iframe, worker), producing 1015 measurement vectors.

The empirical outcomes support the following claims:

- In privacy mode, PAL achieves strong inter-epoch drift on high-entropy surfaces, especially Canvas and WebGL in top and worker contexts.
- In privacy mode, composite strict-equality linkability is eliminated in the top context and sharply reduced in iframe and worker contexts.
- The evaluation pipeline identifies specific context-level gaps (notably worker audio drift) and partial iframe drift, which become concrete future work targets rather than vague limitations.
- Compatibility and privacy modes introduce overhead and increase observed runtime errors relative to vanilla, but vanilla itself produces baseline errors on many sites, so these signals are interpreted as robustness and optimization targets rather than a direct measure of page breakage.

15.5 What This Means Practically

Under the strict-equality attacker model used in this report, PAL demonstrates that an extension-based, deployable approach can significantly reduce fingerprint reusability across time without relying on unstable per-call randomization. This indicates that deterministic drift is a viable design direction for practical privacy tooling that still aims for controlled behavior and debuggable evaluation.

15.6 Next Engineering Milestones

Based on measured gaps, the next engineering milestones are:

- Implement worker audio drift end-to-end and verify that worker audio drift becomes non-zero in privacy mode.
- Improve iframe propagation and drift activation to reduce residual iframe linkability.
- Reduce overhead and variance by caching per-epoch perturbation parameters and minimizing heavy operations in hot hook paths.
- Scale evaluation to a larger site set and report uncertainty estimates for drift and linkability rates.

15.7 Closing Statement

PAL is a technically feasible, deployable system that operationalizes a clear principle: stability within an epoch, change across epochs, applied to high-entropy fingerprint surfaces and evaluated across real execution contexts. The strongest supported claim is that PAL's privacy mode can eliminate strict-equality composite fingerprint linkability across epochs in the top frame and substantially reduce it in iframes and workers. The remaining gaps are concrete, measurable, and directly guide the next iteration of implementation improvements.

CHAPTER 16

RELATED WORK

16.1 Why Related Work Matters Here

PAL sits at the intersection of three families of browser privacy defenses:

- **Randomization-based defenses:** introduce noise or randomness into outputs to reduce repeatability.
- **Standardization-based defenses:** reduce entropy by making outputs more uniform across users.
- **Isolation and partitioning defenses:** restrict or partition state so that identifiers do not carry across sites or contexts.

A credible evaluation requires positioning PAL relative to these families and clarifying what is genuinely different about PAL's approach. PAL is best described as a **controlled deterministic drift** defense: it attempts to preserve stability within epochs, avoid per-call randomness signatures, and change selected surfaces across epochs in a bounded way.

16.2 Fingerprinting Surfaces and Measurement Practices

Fingerprinting is commonly constructed from a combination of graphics, audio, and environment signals:

- **Canvas** based measurement often hashes pixel output from controlled drawings, especially text rendering and shape compositing.
- **WebGL** measurement collects renderer metadata, extension sets, precision characteristics, and render output hashes.
- **Audio** measurement uses OfflineAudioContext pipelines to produce stable arrays that can be hashed.
- **Navigator and Screen** values provide supporting signals and can be used for coarse partitioning and plausibility checks.

PAL's surface selection aligns with the practical observation that graphics and audio surfaces often dominate entropy, while environment signals often act as stabilizers or validation signals in tracker pipelines.

16.3 Randomization and Noise Injection Defenses

Randomization defenses are conceptually simple: add noise so the same machine does not yield the same fingerprint each time. However, there are well-known drawbacks that matter directly for deployability:

- **Per-call randomness signatures:** repeated calls yield fluctuating values. A tracker can detect this by querying the same surface multiple times in a row and checking for instability.
- **Compatibility risk:** many scripts implicitly assume determinism for graphics and audio pipelines. Random noise can break expectations in subtle ways.
- **Evaluation difficulty:** random outcomes complicate debugging and make reproducing results harder.

PAL differs from naive noise injection by enforcing deterministic behavior within an epoch. Drift occurs when the epoch changes, not on every call. This aims to preserve usability characteristics while still disrupting cross-epoch reuse.

16.4 Standardization and Entropy Reduction

Standardization defenses attempt to reduce fingerprint entropy by making many users look the same. This can be effective when:

- the standardized output space is sufficiently small,
- a large number of users share that standardized profile,
- the defense is implemented at a deep enough layer to prevent bypass.

The main challenge is that standardization can create:

- **uniqueness by defense adoption:** if only a small fraction of users adopt the defense, the standardized profile can itself become rare.
- **compatibility pressure:** aggressive standardization can reduce feature detection fidelity and break advanced web applications.

PAL is not purely a standardization tool. It is closer to a controlled variance tool: it changes selected high-entropy surfaces over time, rather than forcing all users into a single static identity.

16.5 Partitioning and Isolation Strategies

Partitioning and isolation strategies aim to prevent identifiers and state from being reused across sites. Common techniques include:

- partitioning caches or storage by top-level site,
- partitioning certain API outputs or making them dependent on the top-level origin,
- restricting high-entropy APIs in third-party contexts.

These approaches can be very strong because they reduce cross-site correlation. However, PAL focuses on a different axis: **cross-time reuse** even within the same site and context. A tracker that repeatedly observes

a user across days can still correlate by strict equality if the fingerprint remains stable. PAL targets this by ensuring that, in privacy mode, the fingerprint is not stable across epochs.

Future versions of PAL can combine these ideas by incorporating optional site-scoped or origin-scoped keys into seed derivation, effectively blending deterministic drift with partitioning.

16.6 Anti-Fingerprinting in Privacy-Focused Browsers

Privacy-focused browsers often implement anti-fingerprinting at a deeper layer than extensions can. This can provide advantages:

- better control over low-level surfaces,
- fewer bypass paths,
- stronger uniformity guarantees.

However, extension-level tools remain valuable for research and rapid iteration because:

- they are deployable without building a full browser fork,
- they allow fast experimentation across different drift and persona policies,
- they can be evaluated and improved incrementally with measurable outcomes.

PAL contributes here as an empirically evaluated extension-level prototype that demonstrates a non-naive drift strategy and quantifies its effect on composite linkability in real crawls.

16.7 How PAL Fits

PAL is best positioned as a controlled, deterministic drift system with explicit evaluation across contexts. Its novelty is not simply that it alters fingerprint surfaces, but that it provides:

- a controlled epoch abstraction,
- deterministic within-epoch stability,
- measured cross-epoch disruption,
- explicit top, iframe, worker evaluation,
- and a central metric focused on strict-equality composite reuse.

16.8 Summary

Related work can be grouped into noise injection, standardization, and partitioning. PAL differs from naive randomization by eliminating per-call jitter and using deterministic drift across epochs. PAL differs from

pure standardization by not aiming for a single static identity, and it differs from pure partitioning by focusing primarily on cross-time reuse rather than only cross-site reuse. The evaluation framework emphasizes composite linkability and multi-context coverage, which makes implementation gaps visible and directly guides future work.

CHAPTER 17

LIMITATIONS AND ETHICAL CONSIDERATIONS

17.1 Scope of Evaluation

The evaluation presented in this report is intentionally scoped. The crawl was conducted on 19 websites, across three epochs and three execution contexts. While the results are internally consistent and sufficient to validate the core design claims of PAL, they do not claim to exhaustively represent the entire web.

The objective of this evaluation is not to produce a final, universal privacy score, but to demonstrate that the proposed mechanisms measurably affect fingerprint stability and linkability under controlled and repeatable conditions. Larger-scale crawls can strengthen statistical confidence, but they are not required to validate the correctness of the underlying mechanism.

17.2 Surface Coverage and Design Tradeoffs

PAL does not attempt to aggressively perturb every observable browser surface. This is a deliberate design choice. Certain low-entropy or highly user-visible properties are preserved to maintain plausibility and reduce breakage risk. As a result, some fingerprint surfaces may remain stable even in privacy mode.

This does not invalidate the privacy objective, because tracking systems rely on composite fingerprints rather than single attributes. The results show that targeted drift on high-entropy surfaces is sufficient to substantially reduce composite linkability, even when some surfaces remain stable.

17.3 Interpretation of Error and Performance Signals

Error events and timing overhead are reported as diagnostic signals rather than as definitive usability metrics. Modern web pages frequently emit runtime errors due to third-party scripts, advertising code, analytics libraries, and experimental features. Therefore, an increase in error counts does not directly imply user-visible failure.

Similarly, measured overhead reflects the cost of instrumentation and evaluation logic in the current prototype. These costs are expected to decrease as the system matures and optimizations are applied. The evaluation is intended to surface these costs early so they can be addressed systematically.

17.4 Ethical Considerations

PAL is designed to improve user privacy by reducing involuntary tracking through fingerprint reuse. It does not attempt to deceive services, impersonate other users, or bypass authentication systems. The system operates entirely within the client environment and modifies only the observable behavior of fingerprint-relevant APIs.

From an ethical standpoint, PAL aligns with user agency and informed consent. Users explicitly select the operational profile (compatibility or privacy), and the system does not interfere with server-side security mechanisms or protected resources. The intent is to restore balance between users and opaque tracking systems by making fingerprint-based identification less reliable.

17.5 Responsible Disclosure and Deployment

As with any browser-level modification, responsible deployment requires careful testing. The results presented here support continued development rather than immediate wide-scale deployment. Future versions should prioritize stability hardening, performance optimization, and transparency so users understand the tradeoffs associated with each operational profile.

—

CHAPTER 18

APPENDICES

CHAPTER A

METRIC DEFINITIONS AND COMPUTATION DETAILS

A.1 Notation

Let:

- $\mathcal{S}$ be the set of sites, with $|\mathcal{S}| = 19$.
- $m \in \{\text{vanilla}, \text{compat}, \text{privacy}\}$ be the mode.
- $e \in \{1, 2, 3\}$ be the epoch identifier.
- $c \in \{\text{top}, \text{iframe}, \text{worker}\}$ be the execution context.
- x be a fingerprint surface (Canvas, WebGL, Audio, Navigator, Screen, Intl).
- $V_{s,e,c,m}(x)$ be the measured output of surface x for site s under epoch e, context c, and mode m.

When the surface output is a complex object (pixel arrays, buffers, etc.), PAL may represent $V(\cdot)$ as a hashed or normalized value. The metric definitions treat these representations as the surface output value used for equality comparisons.

A.2 Inter-Epoch Drift Rate

A.2.1 Definition

For a fixed (s, c, m, x), define an indicator that the surface output differs across two epochs:

$$\Delta_{s,c,m,x}(e_i, e_j) = \begin{cases} 1, & \text{if } V_{s,e_i,c,m}(x) \neq V_{s,e_j,c,m}(x) \\ 0, & \text{otherwise} \end{cases}$$

For $E = 3$ epochs, the epoch pairs are:

$$\mathcal{P} = \{(1, 2), (1, 3), (2, 3)\}$$

The drift rate for a surface x under mode m and context c is:

$$\text{DriftRate}(x, m, c) = \frac{1}{|\mathcal{S}| \cdot |\mathcal{P}|} \sum_{s \in \mathcal{S}} \sum_{(e_i, e_j) \in \mathcal{P}} \Delta_{s,c,m,x}(e_i, e_j)$$

A.2.2 Interpretation

- A drift rate of 0.00 means the surface output is stable across epochs.
- A drift rate of 1.00 means the surface output changes across epochs for every site and every epoch pair.

A.3 Cross-Context Consistency Rate

A.3.1 Definition

For a fixed (s, e, m, x), define an indicator that the surface output matches across contexts:

$$\Gamma_{s,e,m,x} = \begin{cases} 1, & \text{if } V_{s,e,\text{top},m}(x) = V_{s,e,\text{iframe},m}(x) = V_{s,e,\text{worker},m}(x) \\ 0, & \text{otherwise} \end{cases}$$

The cross-context consistency rate for surface x under mode m is:

$$\text{ConsistencyRate}(x, m) = \frac{1}{|\mathcal{S}| \cdot E} \sum_{s \in \mathcal{S}} \sum_{e=1}^{E} \Gamma_{s,e,m,x}$$

A.3.2 Interpretation

- High values indicate surface coherence across contexts within an epoch.
- Low values indicate context-specific differences or incomplete propagation.

A.4 Composite Linkability Rate

A.4.1 Composite Key Construction

Let the composite key be a deterministic function of surface outputs:

$$K_{s,e,c,m} = \mathcal{F}\Big(V_{s,e,c,m}(x_1), V_{s,e,c,m}(x_2), \ldots, V_{s,e,c,m}(x_n)\Big)$$

Where $\mathcal{F}$ is typically concatenation plus hashing. The set $\{x_1, \ldots, x_n\}$ is the set of surfaces included in the composite fingerprint in the evaluation.

A.4.2 Definition

Define a strict link indicator across two epochs:

$$\Lambda_{s,c,m}(e_i, e_j) = \begin{cases} 1, & \text{if } K_{s,e_i,c,m} = K_{s,e_j,c,m} \\ 0, & \text{otherwise} \end{cases}$$

The composite linkability rate for mode m and context c is:

$$\text{Linkability}(m, c) = \frac{1}{|\mathcal{S}| \cdot |\mathcal{P}|} \sum_{s \in \mathcal{S}} \sum_{(e_i, e_j) \in \mathcal{P}} \Lambda_{s,c,m}(e_i, e_j)$$

A.4.3 Interpretation

- 1.00 means the composite fingerprint is fully reusable across epochs under strict equality.
- 0.00 means strict-equality reuse fails across all sites and epoch pairs.

A.5 Error Metrics

A.5.1 Error Event Count

Let $Err_{s,e,c,m}$ be the number of captured runtime error events for site s in epoch e, context c, and mode m. Total error events per mode are:

$$ErrTotal(m) = \sum_{s \in \mathcal{S}} \sum_{e=1}^{E} \sum_{c} Err_{s,e,c,m}$$

A.5.2 Sites with Any Error

Define:

$$HasErr(s, m) = \begin{cases} 1, & \text{if } \sum_{e,c} Err_{s,e,c,m} > 0 \\ 0, & \text{otherwise} \end{cases}$$

Then:

$$ErrSites(m) = \sum_{s \in \mathcal{S}} HasErr(s, m)$$

A.6 Timing Metrics

Let T_i be an individual timing observation in milliseconds for a given (mode, context). The reported statistics are:

$$\mu = \frac{1}{N} \sum_{i=1}^{N} T_i$$

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (T_i - \mu)^2}$$

Where N is the number of timing samples. In this report, mean and standard deviation are reported per (mode, context).

CHAPTER B

EXPERIMENTAL CONFIGURATION AND REPRODUCIBILITY CHECKLIST

B.1 Environment

Fill these values with your exact environment:

- OS: **[FILL: e.g., Windows 10 / Ubuntu 22.04]**
- CPU: **[FILL]**
- RAM: **[FILL]**
- Browser: **[FILL: e.g., Chrome version X.Y]**

B.2 PAL Build and Versioning

- Repository: **PAL (GitHub)**
- Commit hash: **[FILL: git rev-parse HEAD]**
- Extension build method: **[FILL: packed/unpacked]**

B.3 Policy Configuration

- Modes evaluated: vanilla, compat, privacy
- Epoch count: 3
- Epoch definition: **[FILL: time-based / run-based]**
- Surfaces enabled: **[FILL: Canvas, WebGL, Audio, Navigator, Screen, Intl]**
- Context probes enabled: top, iframe, worker

B.4 Crawl Settings

- Sites crawled: 19
- Total measurement vectors: 1015
- Crawl automation tool: **[FILL]**
- Timeouts and retries: **[FILL]**

B.5 Output Artifacts

- Raw logs: JSONL files
- Aggregated metrics: **[FILL: file name(s)]**
- Figures: **[FILL: list figure filenames]**

B.6 Reproduction Steps

1. Install PAL as an unpacked extension.
2. Select the desired mode (vanilla/compat/privacy).
3. Set epoch behavior to 3 epochs.
4. Run the crawler on the 19-site list.
5. Collect JSONL logs.
6. Run the analysis script to compute drift, consistency, linkability, errors, and timing.
7. Generate plots and insert them into the report figures folder.

CHAPTER C

SITE LIST

C.1 Sites Used in the Crawl

Insert your site list here. If you want to anonymize, assign IDs:

Site ID	URL / Description
S01	**[FILL]**
S02	**[FILL]**
S03	**[FILL]**
S04	**[FILL]**
S05	**[FILL]**
S06	**[FILL]**
S07	**[FILL]**
S08	**[FILL]**
S09	**[FILL]**
S10	**[FILL]**
S11	**[FILL]**
S12	**[FILL]**
S13	**[FILL]**
S14	**[FILL]**
S15	**[FILL]**
S16	**[FILL]**
S17	**[FILL]**
S18	**[FILL]**
S19	**[FILL]**

CHAPTER D

ERROR TAXONOMY AND INTERPRETATION

D.1 Why a Taxonomy Helps

Raw error counts are noisy because modern sites generate baseline errors even without any privacy tool. A taxonomy helps separate:

- errors likely unrelated to PAL (third-party noise),
- errors plausibly introduced or amplified by hook behavior,
- errors that indicate real compatibility risk.

D.2 Proposed Error Categories

D.2.1 E1: Baseline Third-Party Script Errors

These are errors originating from analytics, ads, and third-party widgets that are known to produce intermittent failures. They often occur in vanilla mode as well.

D.2.2 E2: API Contract Violations

These are errors that suggest a wrapper returned an unexpected type or shape, or violated expected invariants. These are the highest priority errors for hardening.

D.2.3 E3: Security Exceptions

These include exceptions such as permission errors, cross-origin restrictions, or sandbox restrictions that may be triggered when probing surfaces inside restricted contexts.

D.2.4 E4: Feature-Detection Mismatches

These occur when scripts feature-detect one behavior, but the environment returns a value inconsistent with the detected capability.

D.2.5 E5: CSP and Policy Violations

CSP violations (when captured) indicate sites blocking injected scripts or restricting execution. These are relevant to iframe propagation.

D.3 How to Use the Taxonomy

For each captured error line in the logs, classify it as one of E1-E5. Then report:

- total errors per category per mode,
- number of sites impacted per category,
- the top recurring error messages.

This framing makes the error section more informative and prevents over-interpreting raw counts.

D.4 What This Report Currently Shows

In the current evaluation, total error events were:

- Vanilla: 42
- Compat: 73
- Privacy: 72

Because vanilla already exhibits errors on 12 of 19 sites, the most defensible interpretation is that errors are an engineering signal, not a direct measure of breakage. The taxonomy approach above is a structured next step to strengthen this section further.

—

CHAPTER E

REFERENCES

- Eckersley, P. (2010). How Unique Is Your Web Browser? *Proceedings of the Privacy Enhancing Technologies Symposium*.
- Laperdrix, P., Rudametkin, W., & Baudry, B. (2016). Beauty and the Beast: Diverting Modern Web Browsers to Build Unique Fingerprints. *IEEE Symposium on Security and Privacy*.
- Mozilla Research. (2019). Fingerprinting Resistance in Firefox.
- Brave Software. (2020). Privacy Budget and Fingerprinting Protections.

BIBLIOGRAPHY

[1] The Tor Project, *Tor Browser Design and Fingerprinting Resistance*, Online documentation.

[2] Brave Software, *Brave Anti-Fingerprinting Protections*, Online documentation.

[3] Google Chrome Developers, *Manifest V3 Documentation*, Online documentation.